

04. Ggplot2

Módulo 5. Inteligencia Artificial

Jorge
Juvenal
Campos Ferreira

✉ juvenal.campos@tec.mx

Reestructurar datos

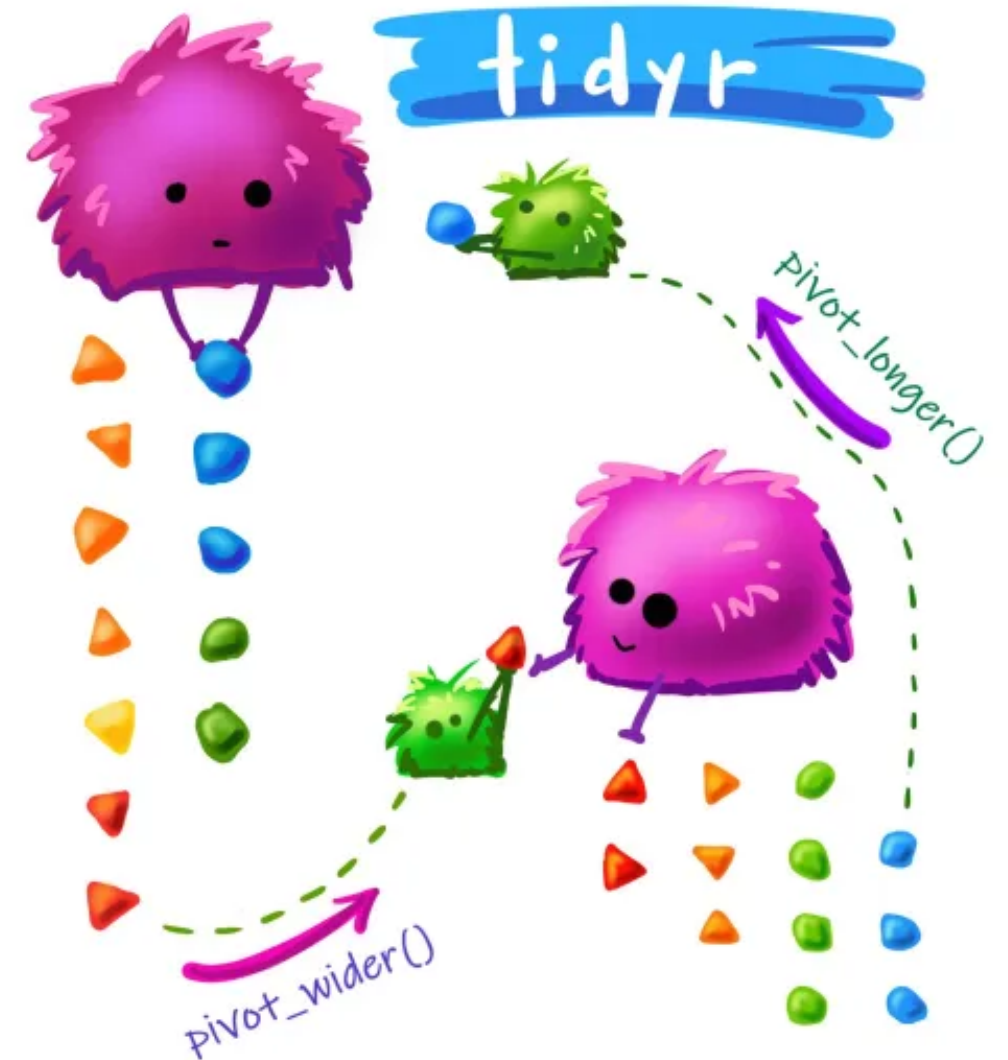
A veces los datos no llegan en el formato que necesitas. Las dos operaciones más comunes son pivotar entre formato largo y formato ancho:

wide

id	x	y	z
1	a	c	e
2	b	d	f

long

id	key	val
1	x	a
2	x	b
1	y	c
2	y	d
1	z	e
2	z	f



Revisar: <https://juvenalcampos.com/posts/2020-12-04-pivoteando-bases/>



pivot_longer()

country	1999	2000	2001	2002
Angola	800	750	925	1020
India	20100	25650	26800	27255
Mongolia	450	512	510	586

Pivot data longer

```
data %>%  
  pivot_longer(  
    cols = 1999:2002,  
    names_to = "year",  
    values_to = "cases"  
  )
```



country	year	cases
Angola	1999	800
Angola	2000	750
Angola	2001	925
Angola	2002	1020
India	1999	20100
India	2000	25650
India	2001	26800
India	2002	27255
Mongolia	1999	450
Mongolia	2000	512
Mongolia	2001	510
Mongolia	2002	586

pivot_wider()

country	year	metric
x	1960	10
x	1970	13
x	2010	15
y	1960	20
y	1970	23
y	2010	25
z	1960	30
z	1970	33
z	2010	35

`pivot_wider(names_from = "year",
names_prefix = "yr",
values_from = "metric")`

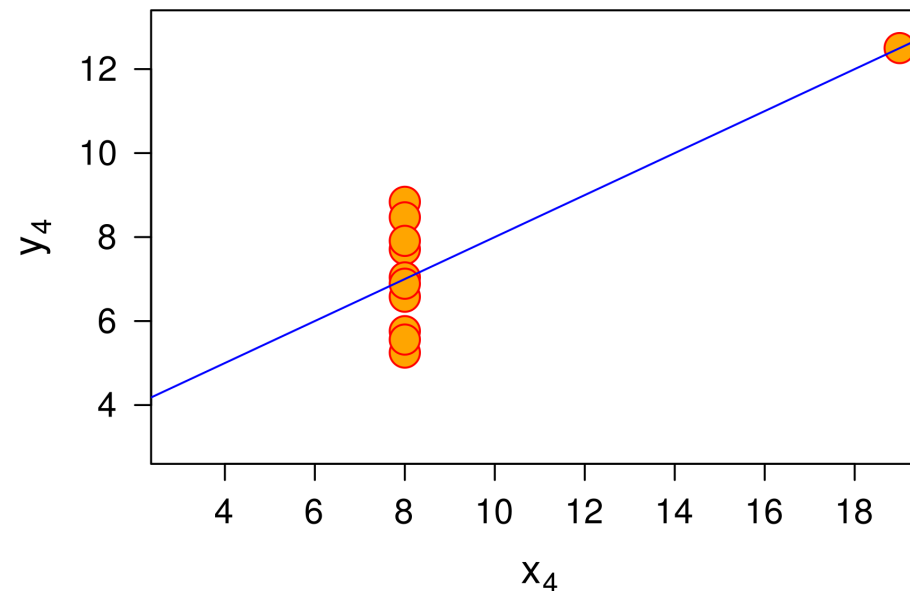
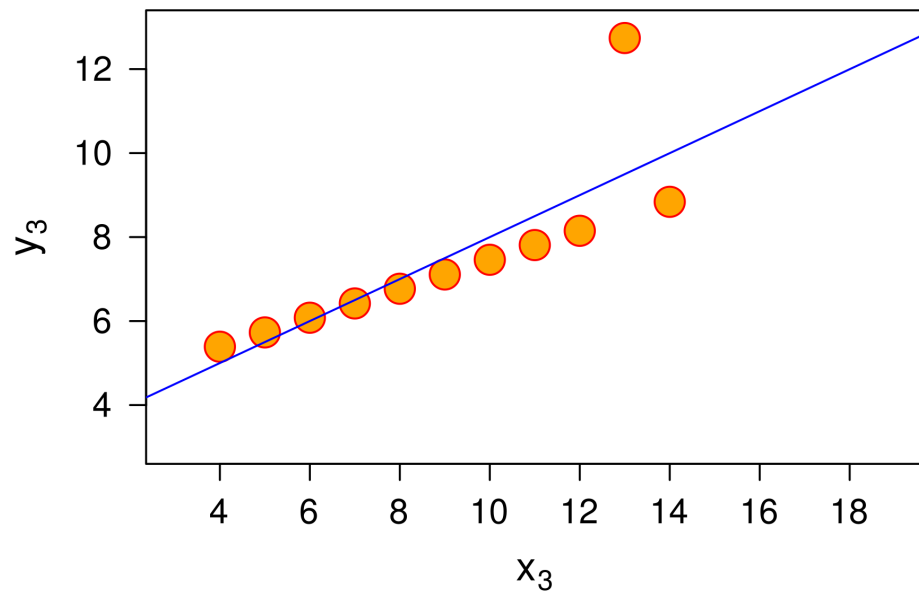
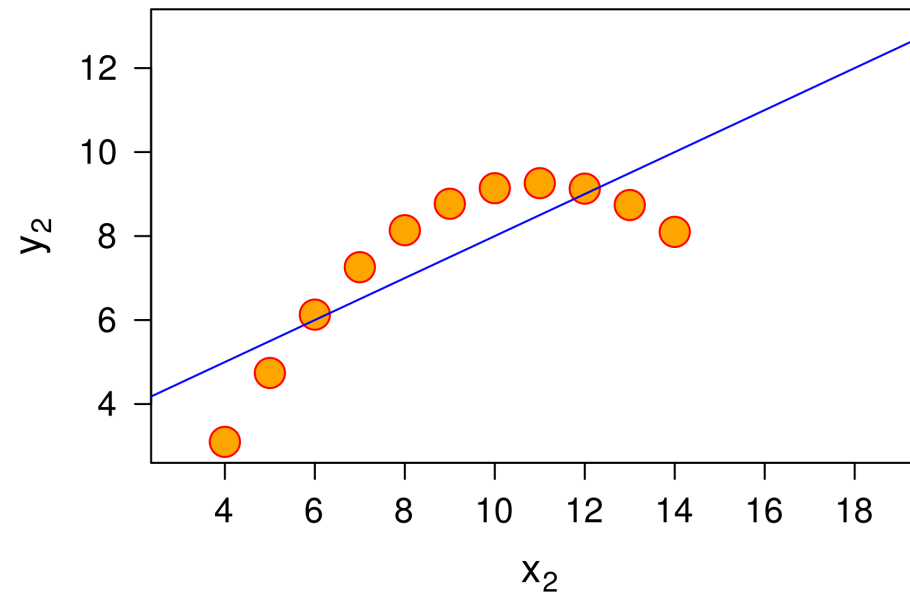
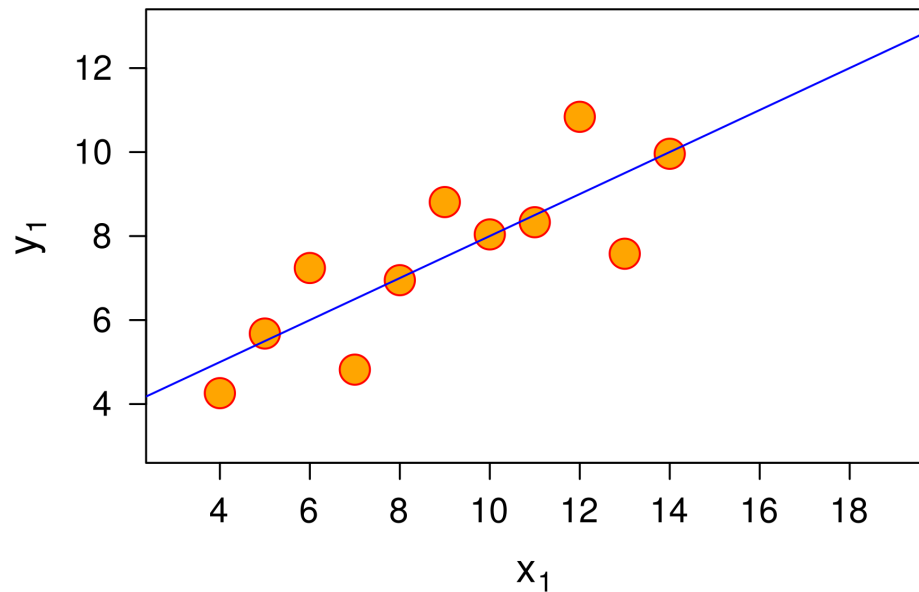
country	yr1960	yr1970	yr2010
x	10	13	15
y	20	23	25
z	30	33	35

`pivot_longer(cols = yr1960:yr2010,
names_to = "year",
names_prefix = "yr",
values_to = "metric")`

Ejercicio 01

- En la carpeta de ejercicio 3, cargue los datos provenientes de INEGI (PIB trimestral) SIN modificar la fuente primaria.
- Pase la tabla a formato largo
- ¿Cuál es el sector que más ha crecido desde el **2019T4 (antes de la pandemia)**?

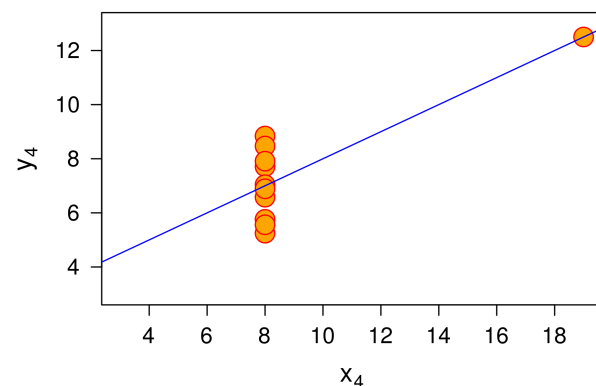
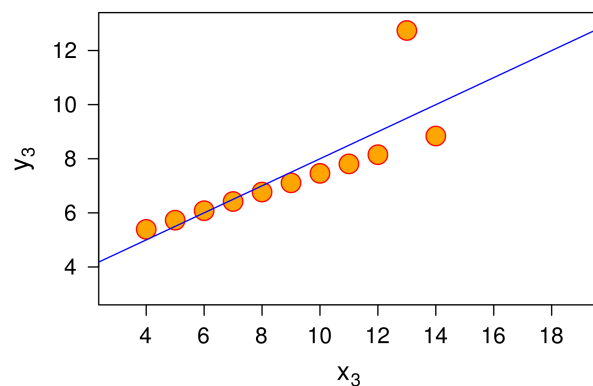
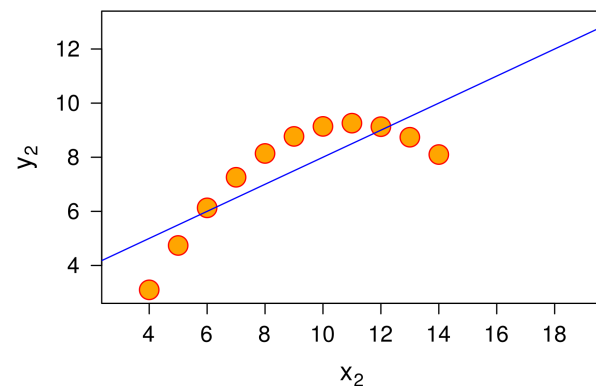
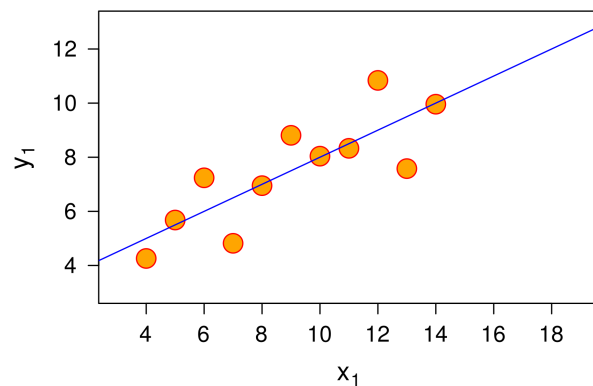
¿Por qué visualizar?



- * Misma media de X e Y
- * Misma varianza,
- * Misma correlación ($r = 0.816$),
- * Misma recta de regresión.

Y sin embargo, al *graficarlos* se ven completamente **diferentes**: uno es **lineal**, otro es **curvo**, otro tiene un **outlier**, y otro tiene una **estructura completamente distinta**.

¿Por qué visualizar?



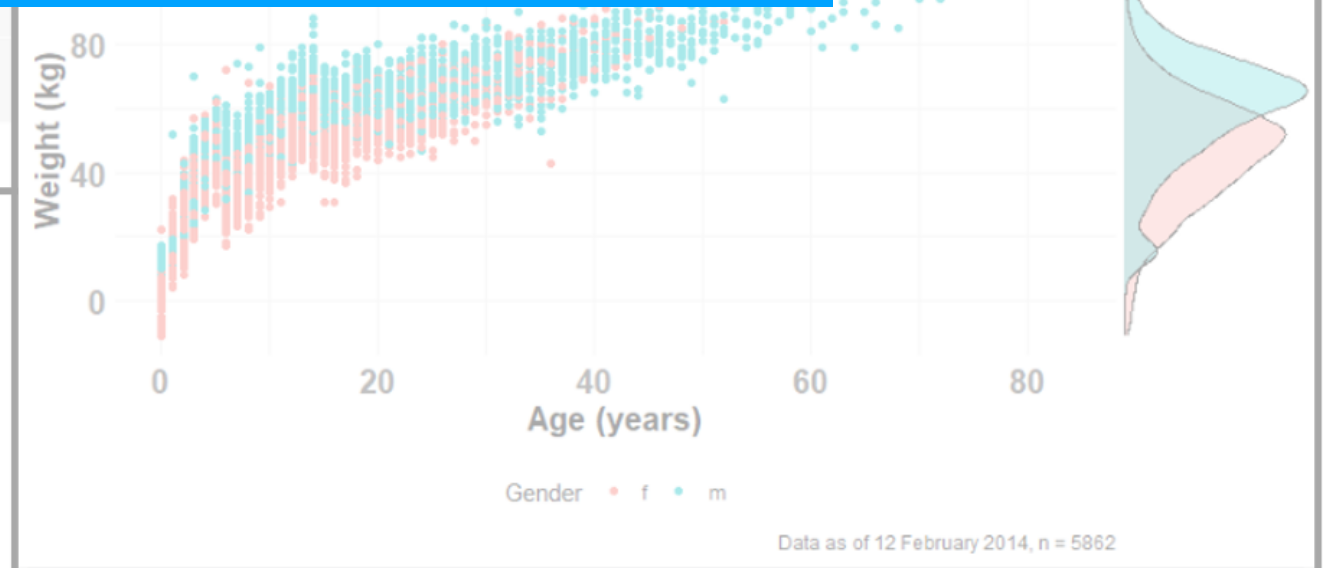
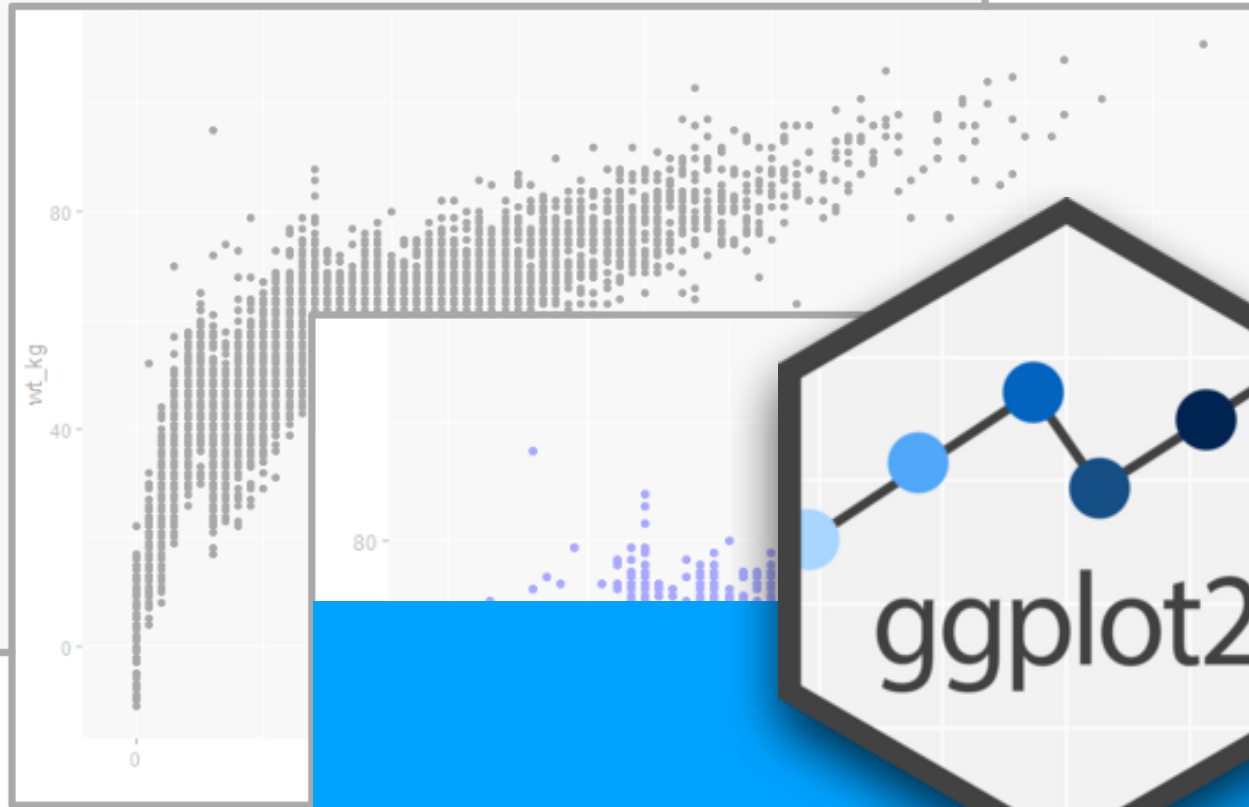
Lección del cuarteto de Anscombe

*Nunca confíes solo en las estadísticas descriptivas. **Trata de graficar tus datos antes de modelar.** Un gráfico de dispersión simple puede revelar patrones (no linealidad, outliers, agrupamientos) que la media y la correlación ocultan completamente.*





ggplot



Data as of 12 February 2014, n = 5862

¿Por qué ggplot?

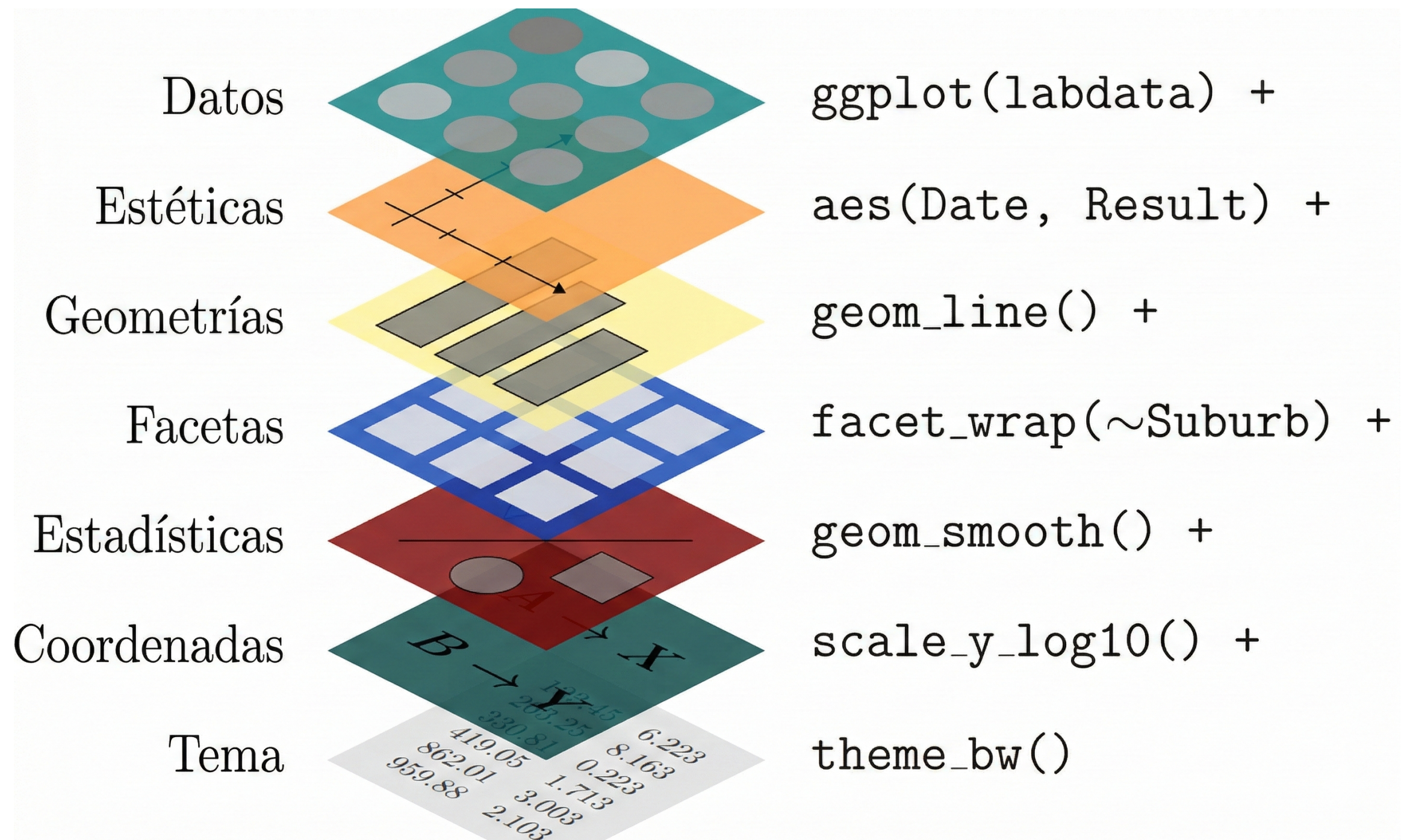


ggplot2 es el paquete de visualización más usado en R y uno de los más influyentes en la historia de la visualización de datos.

Fue creado por *Hadley Wickham* basándose en la *gramática de gráficos* (Grammar of Graphics, **gg**) de Leland Wilkinson.

La idea central: un gráfico es una composición de capas con una gramática definida, no una colección de tipos de gráfico predefinidos.

¿Por qué ggplot?

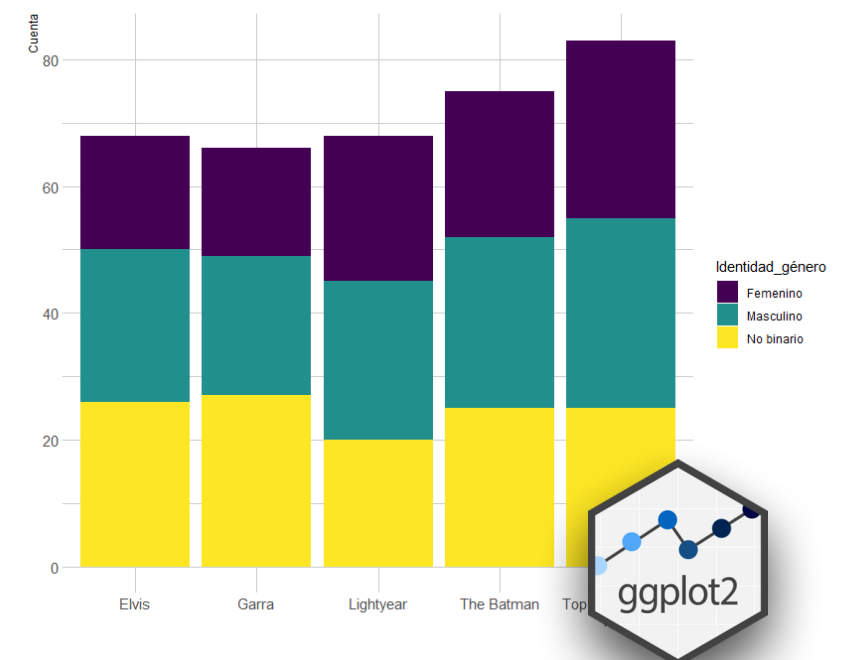
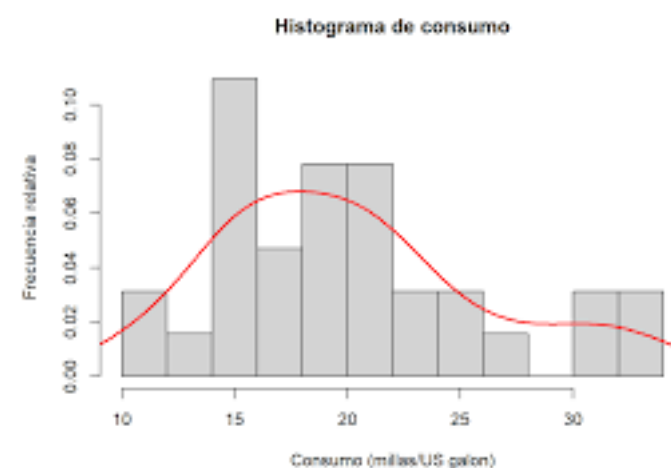
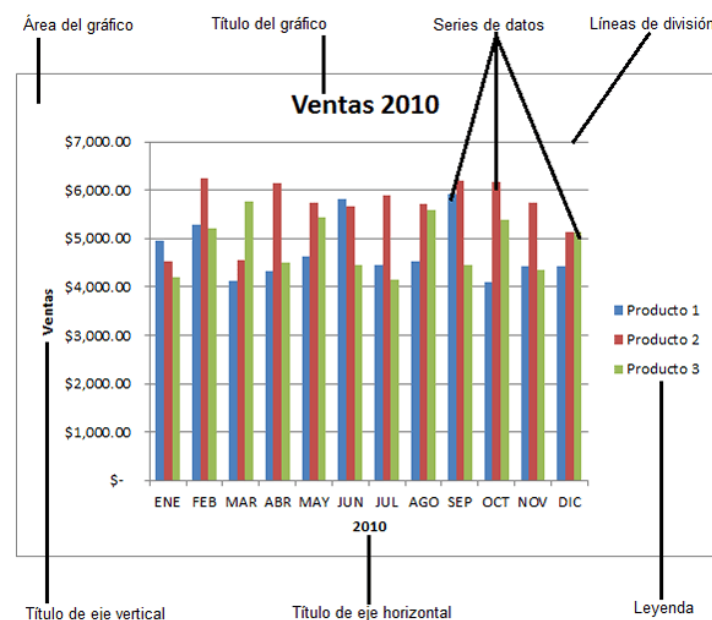


¿Por qué ggplot?



Ventajas de ggplot2 frente a los gráficos base de R y frente a Excel:

- **Consistencia:** todos los gráficos se construyen con la misma sintaxis. Aprendes una vez, aplicas siempre.
- **Composición por capas:** puedes agregar elementos progresivamente (datos + geometría + color + facetas + tema).
- **Calidad profesional:** los gráficos son publicables directamente en revistas y reportes.
- **Integración con dplyr:** puedes encadenar manipulación y visualización con el pipe: `datos %>% filter(...) %>% ggplot(...)`.



Componentes mínimos



Todo gráfico en ggplot2 tiene tres componentes **mínimos**:

1. Datos (data)

Data frame o tibble con la información a graficar en el formato adecuado

2. Geometría (geom)

Decidir como representar los datos gráficamente: puntos, líneas, barras, histogramas

3. Estética (aes)

Mapeos visuales: X, Y, color, tamaño, forma. (Cómo representar las variables de la tabla en la gráfica).

Opcionales

Facetas, temas, etiquetas, escalas personalizadas

Analogía: construir un gráfico como construir una oración

Un gráfico en ggplot2 se construye como una oración gramatical: el SUJETO son los datos, el VERBO es la geometría (qué forma visual), y los COMPLEMENTOS son las estéticas (qué variable va dónde, qué color usar). Así como puedes agregar adjetivos y adverbios a una oración, puedes agregar capas de personalización al gráfico.



Visualización de datos con ggplot2 : : GUÍA RÁPIDA



Básico

ggplot2 se basa en la gramática de los gráficos, la idea de que se pueden construir todos los gráficos a partir de los mismos componentes: un conjunto de datos, un sistema de coordenadas y geoms, marcas visuales que representan puntos de datos.



Para mostrar valores, asigne variables de los datos a propiedades visuales del geom (estética) como el tamaño, el color y las ubicaciones x e y.



Complete la siguiente plantilla para crear un gráfico.

ggplot (data = <DATOS>) +
<FUNCIÓN_GEOM> (mapping = aes(<MAPEADO>),
stat = <STAT>, position = <POSICIÓN>) +
<FUNCIÓN_COORDENADAS> +
<FUNCIÓN_FACETADO> +
<FUNCIÓN_ESCALA> +
<FUNCIÓN_TEMA>

requerido
No requerido, sensible predeterminados suministrados

ggplot(data = mpg, aes(x = cty, y = hwy)) Comienza un gráfico al que se termina añadiendo capas. Agregar una función geom por capa.

last_plot() Devuelve la última gráfica.

ggsave("plot.png", width = 5, height = 5) Guarda el último gráfico como un archivo de 5' x 5' llamado "plot.png" en el directorio de trabajo. Coincide el tipo de archivo con la extensión del archivo.

Aes Valores estéticos comunes.

color y fill - texto ("red", "#RRGGBB")

linetype - entero o texto (0 = "blank", 1 = "solid", 2 = "dashed", 3 = "dotted", 4 = "dotdash", 5 = "longdash", 6 = "twodash")

size - enterp (en mm para el tamaño de los puntos y el texto)

linewidth - entero (en mm para el ancho de líneas)

shape - entero/nombre de la forma o un carácter ("a")



Geoms

Utilice una función geom para representar puntos de datos, utilice las propiedades estéticas del geom para representar variables. Cada función devuelve una capa.

GRÁFICAS PRIMITIVAS

a <- ggplot(economics, aes(date, unemploy))
b <- ggplot(seals, aes(x = long, y = lat))

a + geom_blank() y **a + expand_limits()**
Asegúrese de que los límites incluyan valores en todas las gráficas.

b + geom_curve(aes(yend = lat + 1, xend = long + 1), curvature = 1) - x, xend, y, yend, alpha, angle, color, curvature, linetype, size

a + geom_path(lineend = "butt", linejoin = "round", linemitre = 1) - x, y, alpha, color, group, linetype, size

a + geom_polygon(aes(alpha = 50)) - x, y, alpha, color, fill, group, subgroup, linetype, size

b + geom_rect(aes(xmin = long, ymin = lat, xmax = long + 1, ymax = lat + 1)) - xmax, xmin, ymax, ymin, alpha, color, fill, linetype, size

a + geom_ribbon(aes(ymin = unemploy - 900, ymax = unemploy + 900)) - x, ymax, ymin, alpha, color, fill, group, linetype, size

SEGMENTOS DE LÍNEA

Estéticas común: x, y, alpha, color, linetype, size

b + geom_abline(aes(intercept = 0, slope = 1))
b + geom_hline(aes(yintercept = lat))
b + geom_vline(aes(xintercept = long))

b + geom_segment(aes(yend = lat + 1, xend = long + 1))
b + geom_spoke(aes(angle = 1:155, radius = 1))

UNA VARIABLE continua

c <- ggplot(mpg, aes(hwy)); c2 <- ggplot(mpg)

c + geom_area(stat = "bin") - x, y, alpha, color, fill, linetype, size

c + geom_density(kernel = "gaussian") - x, y, alpha, color, fill, group, linetype, size, weight

c + geom_dotplot() - x, y, alpha, color, fill

c + geom_freqpoly() - x, y, alpha, color, group, linetype, size

c + geom_histogram(binwidth = 5) - x, y, alpha, color, fill, linetype, size, weight

c2 + geom_qq(aes(sample = hwy)) - x, y, alpha, color, fill, linetype, size, weight

discreta

d <- ggplot(mpg, aes(f))

d + geom_bar() - x, alpha, color, fill, linetype, size, weight

DOS VARIABLES

ambas continuas

e <- ggplot(mpg, aes(cty, hwy))

e + geom_label(aes(label = cty), nudge_x = 1, nudge_y = 1) - x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

e + geom_point() - x, y, alpha, color, fill, shape, size, stroke

e + geom_quantile() - x, y, alpha, color, group, linetype, size, weight

e + geom_rug(sides = "bl") - x, y, alpha, color, linetype, size

e + geom_smooth(method = lm) - x, y, alpha, color, fill, group, linetype, size, weight

e + geom_text(aes(label = cty), nudge_x = 1, nudge_y = 1) - x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

una discreta, una continua

f <- ggplot(mpg, aes(class, hwy))

f + geom_col() - x, y, alpha, color, fill, group, linetype, size

f + geom_boxplot() - x, y, lower, middle, upper, ymax, ymin, alpha, color, fill, group, linetype, shape, size, weight

f + geom_dotplot(binaxis = "y", stackdir = "center") - x, y, alpha, color, fill, group

f + geom_violin(scale = "area") - x, y, alpha, color, fill, group, linetype, size, weight

ambas discretas

g <- ggplot(diamonds, aes(cut, color))

g + geom_count() - x, y, alpha, color, fill, shape, size, stroke

e + geom_jitter(height = 2, width = 2) - x, y, alpha, color, fill, shape, size

TRES VARIABLES

seals\$z <- with(seals, sqrt(delta_long^2 + delta_lat^2)); l <- ggplot(seals, aes(long, lat))

l + geom_contour(aes(z = z)) - x, y, z, alpha, color, group, linetype, size, weight

l + geom_contour_filled(aes(fill = z)) - x, y, alpha, color, fill, group, linetype, size, subgroup

distribución bivariada continua

h <- ggplot(diamonds, aes(carat, price))

h + geom_bin2d(binwidth = c(0.25, 500)) - x, y, alpha, color, fill, linetype, size, weight

h + geom_density_2d() - x, y, alpha, color, group, linetype, size

h + geom_hex() - x, y, alpha, color, fill, size

función continua

i <- ggplot(economics, aes(date, unemploy))

i + geom_area() - x, y, alpha, color, fill, linetype, size

i + geom_line() - x, y, alpha, color, group, linetype, size

i + geom_step(direction = "hv") - x, y, alpha, color, group, linetype, size

visualización de error

df <- data.frame(grp = c("A", "B"), fit = 4:5, se = 1:2)
j <- ggplot(df, aes(grp, fit, ymin = fit - se, ymax = fit + se))

j + geom_crossbar(fatten = 2) - x, y, ymax, ymin, alpha, color, fill, group, linetype, size

j + geom_errorbar() - x, ymax, ymin, alpha, color, group, linetype, size, width
Also **geom_errorbarh()**.

j + geom_linerange() - x, ymin, ymax, alpha, color, group, linetype, size

j + geom_pointrange() - x, y, ymin, ymax, alpha, color, fill, group, linetype, shape, size

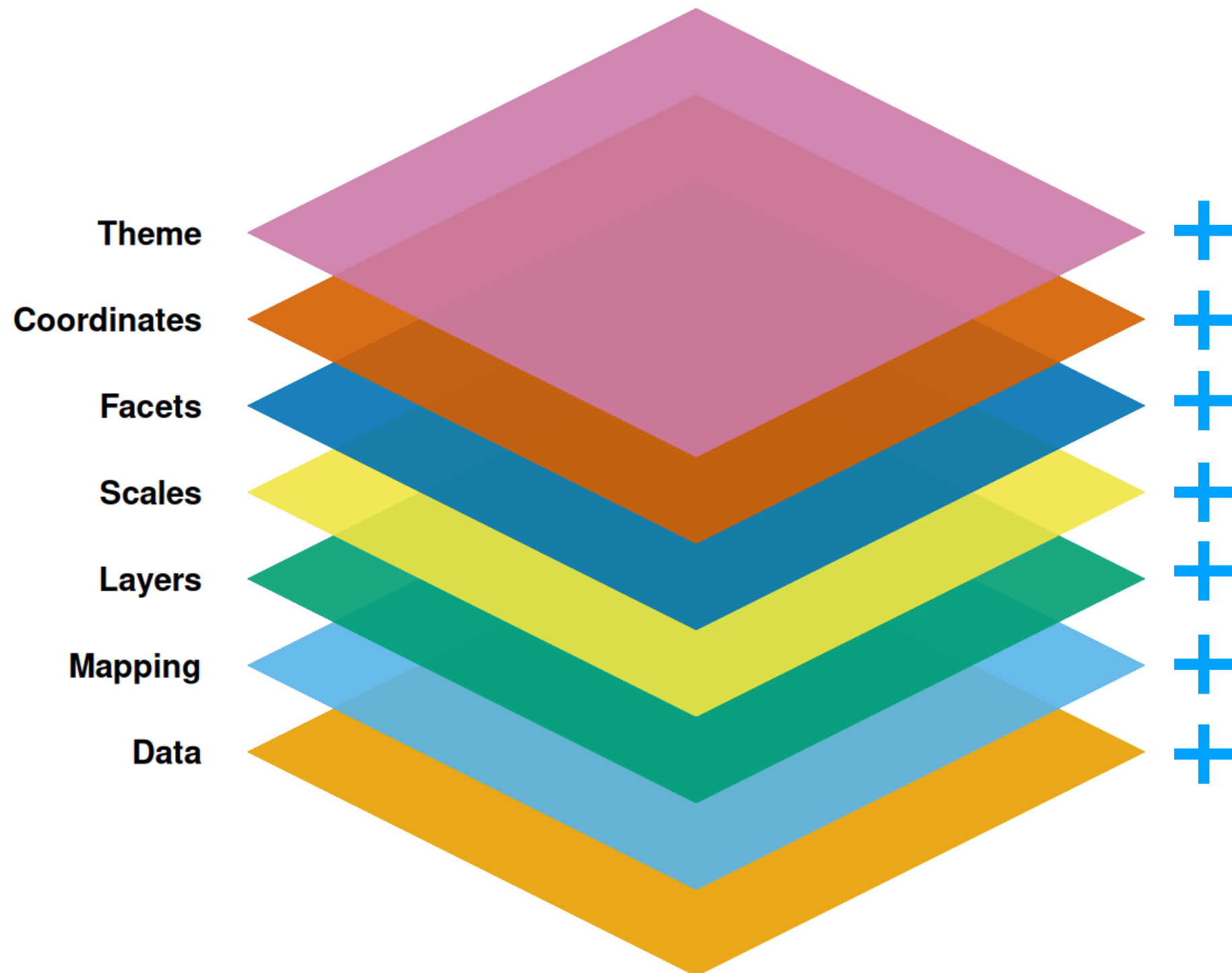
mapas

Dibuje el objeto geométrico apropiado en función de las características simples presentes en los argumentos de data. aes():
map_id, alpha, color, fill, linetype, linewidth.

nc <- sf::st_read(system.file("shape/nc.shp", package = "sf"))

ggplot(nc) + geom_sf(aes(fill = AREA))

En ggplot vamos sumando capas



No concatenando



(El símbolo “+” hace esa función)

Ejemplo de ggplot

Dados los siguientes datos:

```
library(tidyverse)

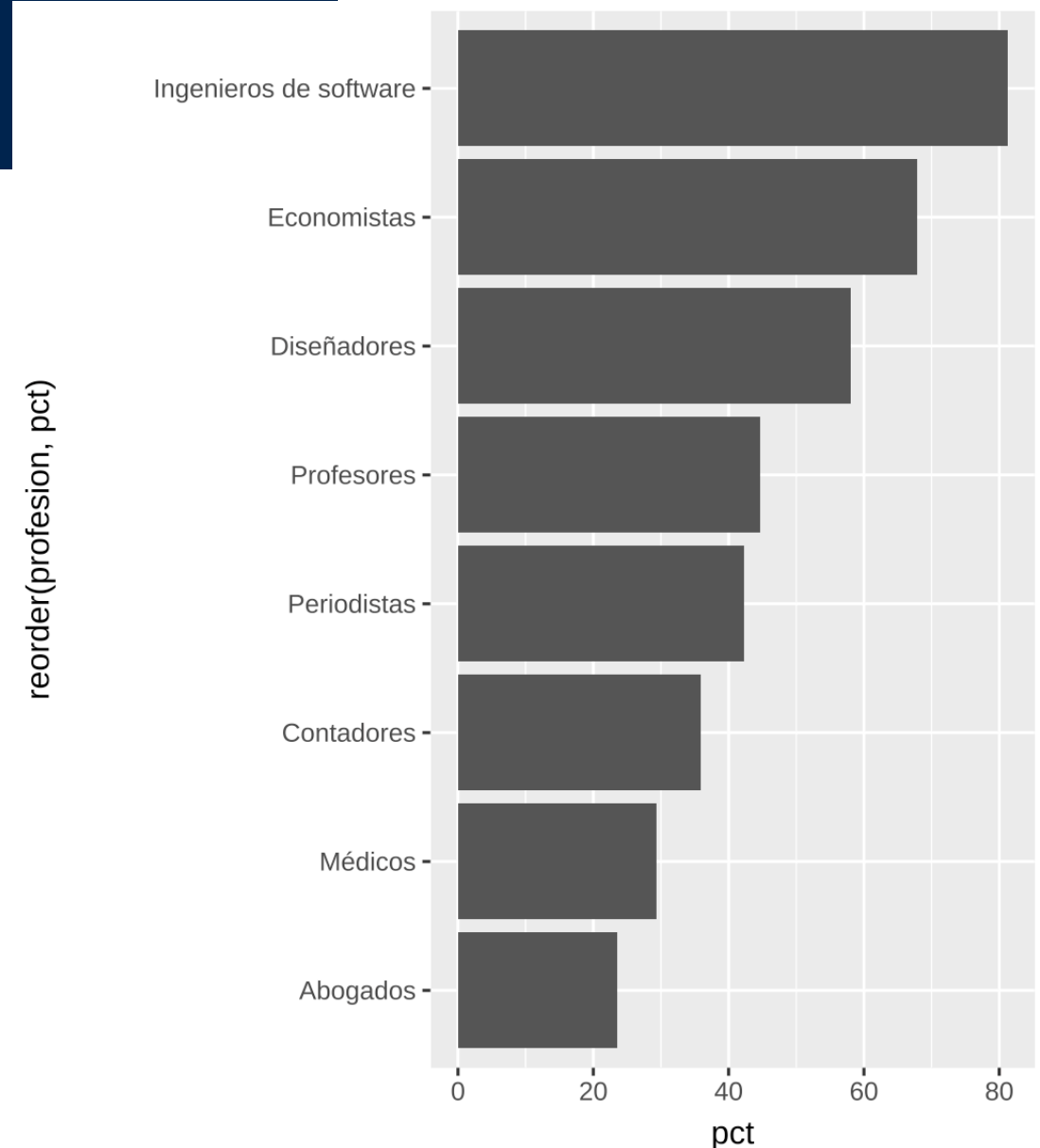
# Datos
datos <- tribble(
  ~profesion,      ~pct,    ~pct_low, ~pct_upp,
  "Periodistas",   42.3,    38.1,    46.5,
  "Economistas",   67.8,    63.2,    72.4,
  "Diseñadores",   58.1,    53.5,    62.7,
  "Abogados",      23.6,    19.8,    27.4,
  "Ingenieros de software", 81.2,    77.4,    85.0,
  "Contadores",    35.9,    31.7,    40.1,
  "Médicos",       29.4,    25.3,    33.5,
  "Profesores",    44.7,    40.2,    49.2
) %>%
mutate(es_max = pct == max(pct))
```

Ejemplo de ggplot

Fase 1. Esqueleto:

```
# --- Etapa 1: Esqueleto ---  
p1 <- ggplot(datos, aes(x = reorder(profesion, pct), y = pct)) +  
  geom_col() +  
  coord_flip()
```

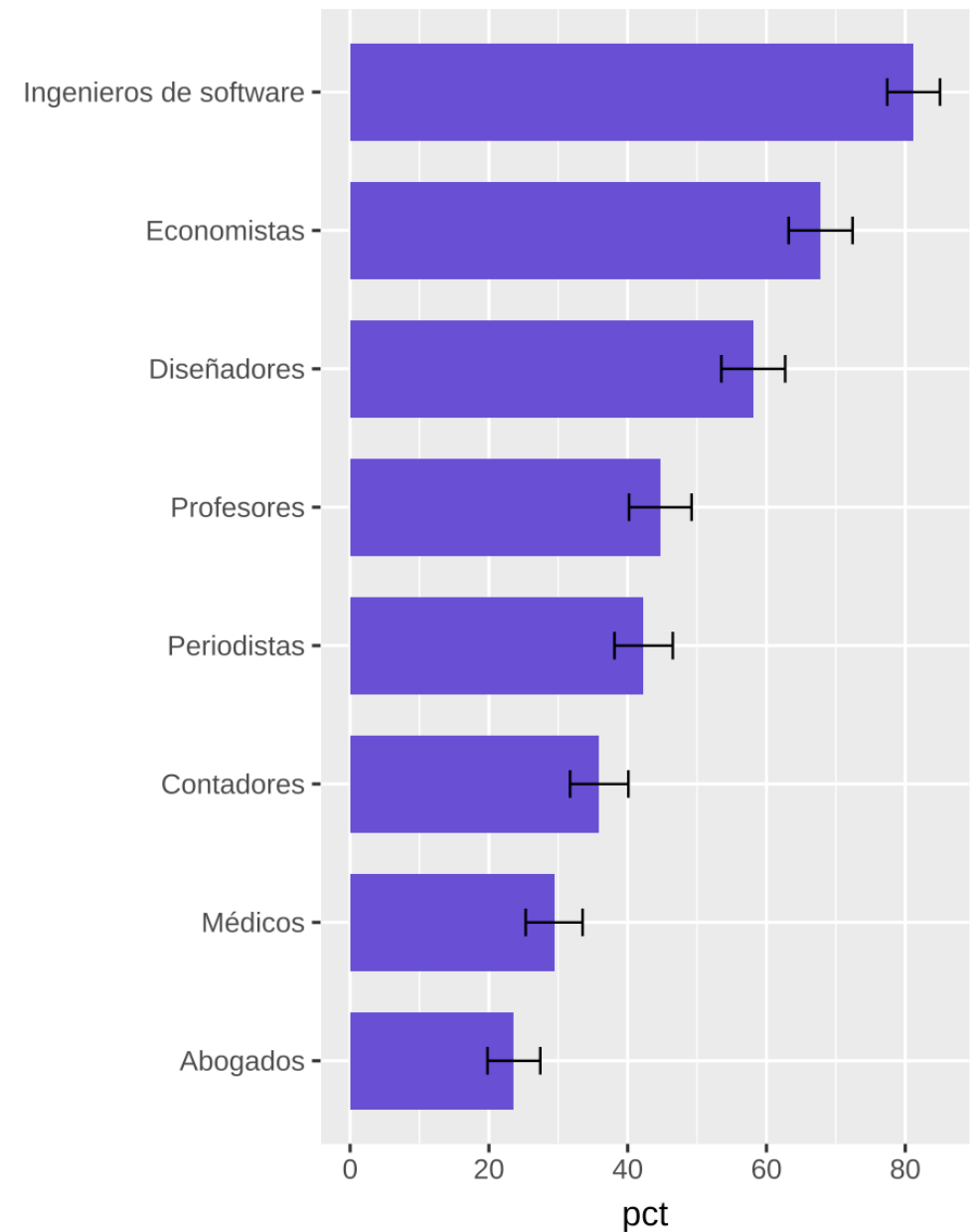
p1



Ejemplo de ggplot

```
# --- Etapa 2: Color y error bars ---  
p2 <- ggplot(datos, aes(x = reorder(profesion, pct), y = pct)) +  
  geom_col(fill = "#6950d8", width = 0.7) +  
  geom_errorbar(  
    aes(ymin = pct_low, ymax = pct_upp),  
    width = 0.2,  
    linewidth = 0.4  
  ) +  
  coord_flip()  
p2
```

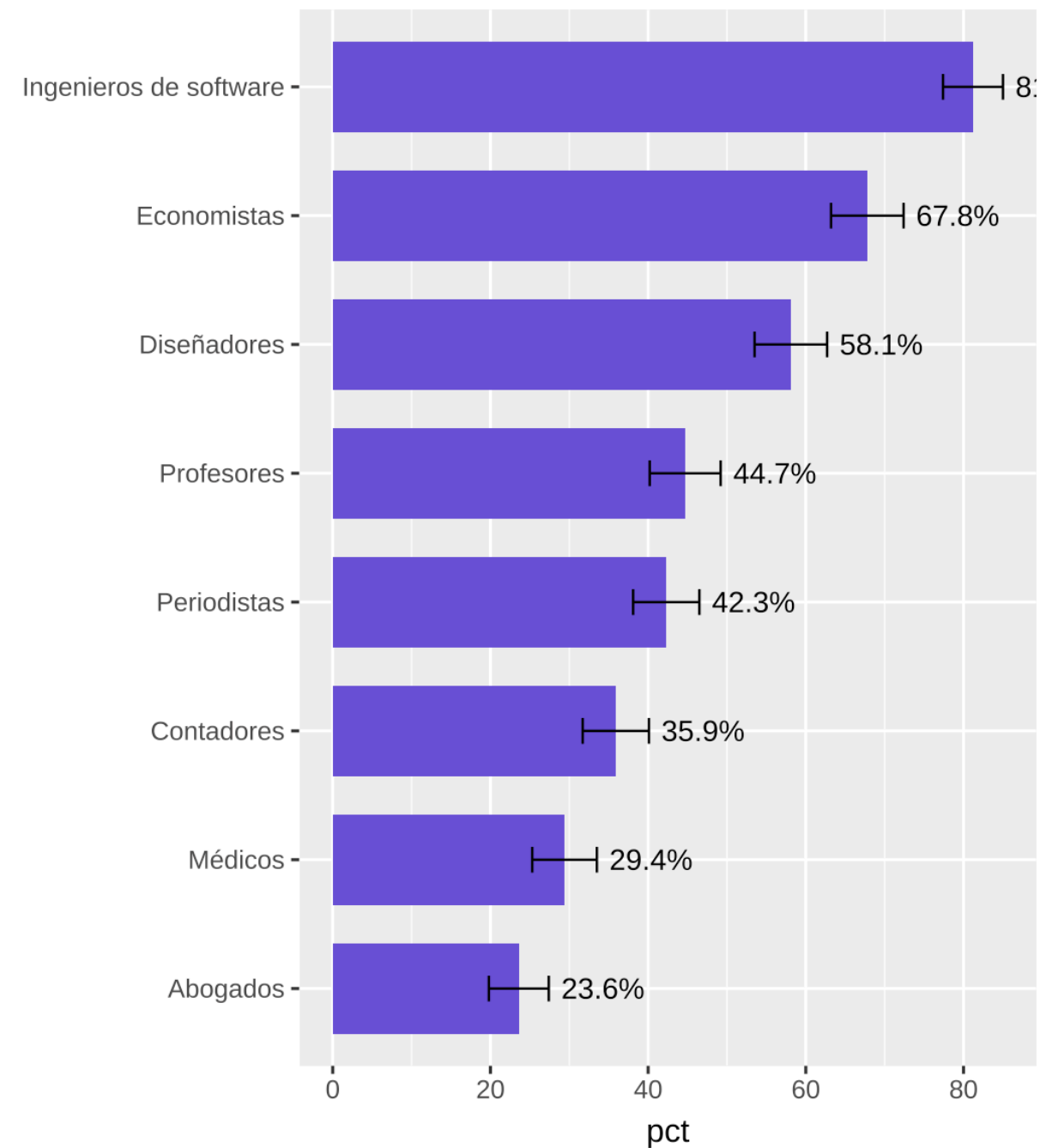
reorder(profesion, pct)



Ejemplo de ggplot

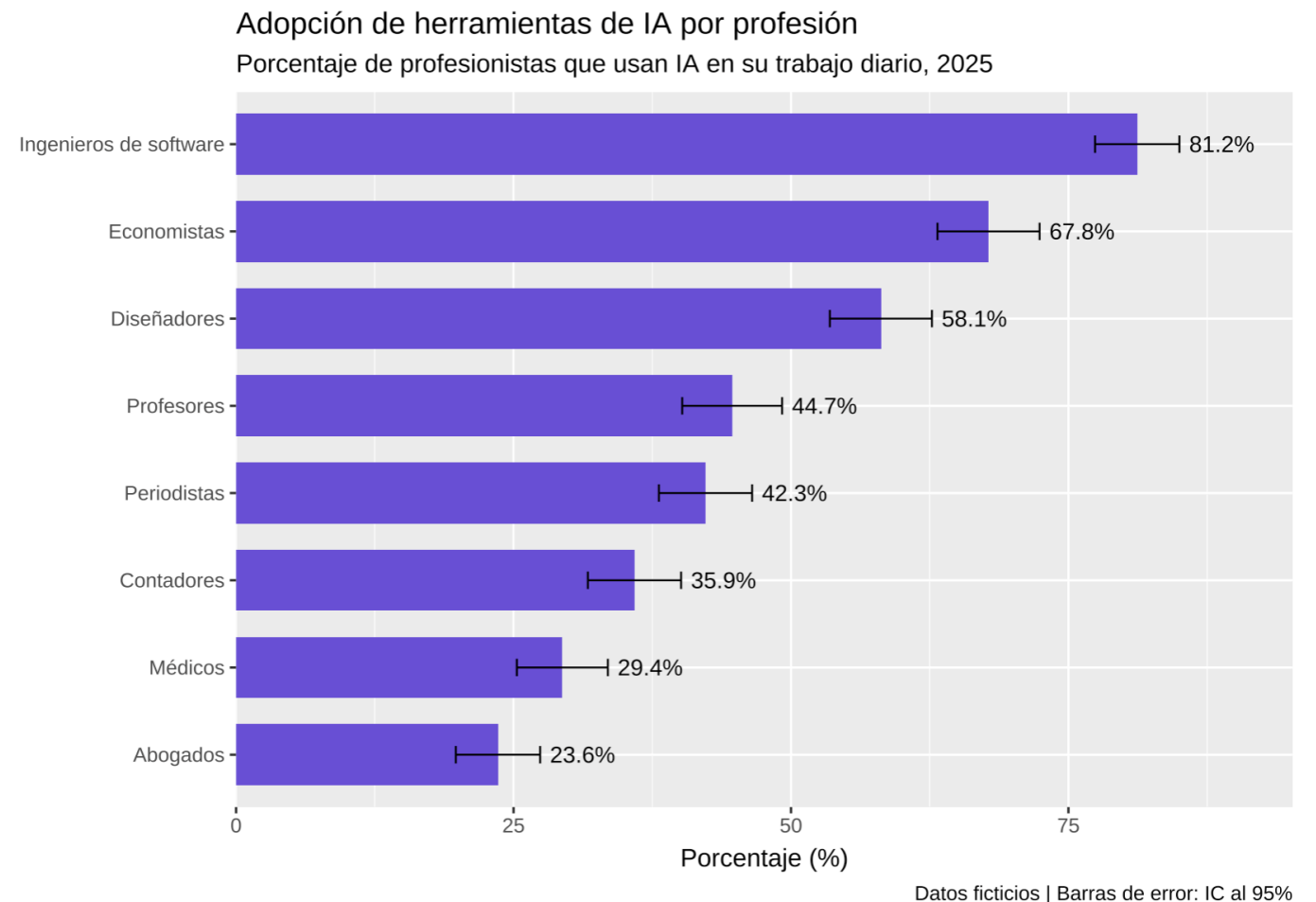
```
# --- Etapa 3: Etiquetas de texto ---
p3 <- ggplot(datos, aes(x = reorder(profesion, pct), y = pct)) +
  geom_col(fill = "#6950d8", width = 0.7) +
  geom_errorbar(
    aes(ymin = pct_low, ymax = pct_upp),
    width = 0.2,
    linewidth = 0.4
  ) +
  geom_text(
    aes(y = pct_upp, label = paste0(pct, "%")),
    hjust = -0.15,
    size = 3.5
  ) +
  coord_flip()
p3
```

reorder(profesion, pct)



Ejemplo de ggplot

```
# --- Etapa 4: Títulos y espacio ---
p4 <- ggplot(datos,
             aes(x = reorder(profesion, pct),
                 y = pct)) +
  geom_col(fill = "#6950d8",
           width = 0.7) +
  geom_errorbar(
    aes(ymin = pct_low,
        ymax = pct_upp),
    width = 0.2,
    linewidth = 0.4
  ) +
  geom_text(
    aes(y = pct_upp,
        label = paste0(pct, "%")),
    hjust = -0.15,
    size = 3.5
  ) +
  coord_flip() +
  scale_y_continuous(expand =
                     expansion(mult = c(0, 0.12))) +
  labs(
    title = "Adopción de herramientas de IA por profesión",
    subtitle = "Porcentaje de profesionistas que usan IA en su trabajo diario, 2025",
    x = NULL,
    y = "Porcentaje (%)",
    caption = "Datos ficticios | Barras de error: IC al 95%"
  )
p4
```

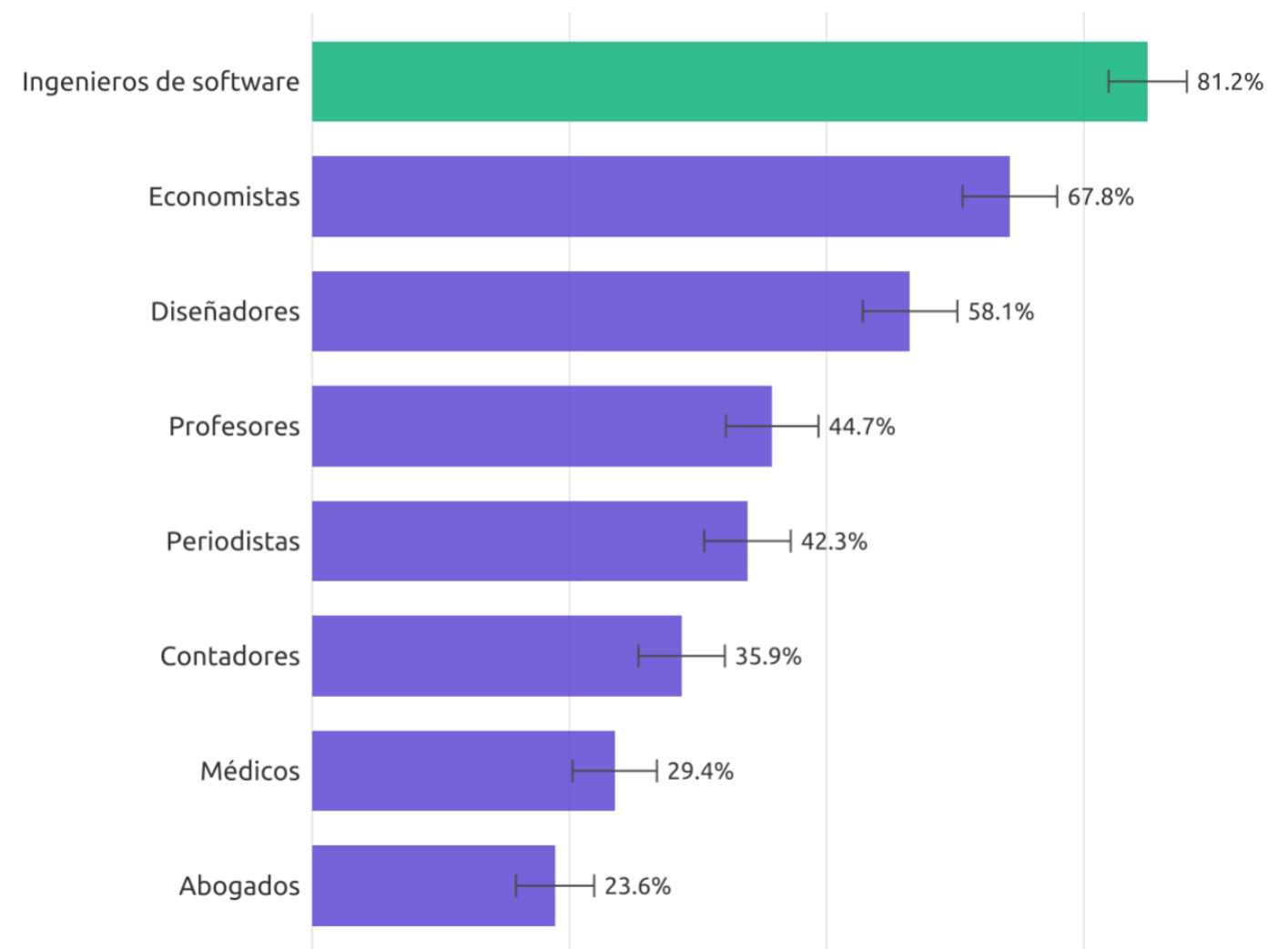


Ejemplo de ggplot

```
# --- Etapa 7: Highlight de la barra más alta ---
p7 <- ggplot(datos, aes(x = reorder(profesion, pct), y = pct)) +
  geom_col(aes(fill = es_max), width = 0.7, alpha = 0.9, show.legend = FALSE) +
  geom_errorbar(
    aes(ymin = pct_low, ymax = pct_upp),
    width = 0.2,
    linewidth = 0.4,
    color = "gray30"
  ) +
  geom_text(
    aes(y = pct_upp, label = paste0(pct, "%")),
    hjust = -0.15,
    size = 3.5,
    family = "Ubuntu",
    color = "gray20"
  ) +
  coord_flip(clip = "off") +
  scale_fill_manual(values = c("FALSE" = "#6950d8", "TRUE" = "#00b783")) +
  scale_y_continuous(expand = expansion(mult = c(0, 0.12))) +
  labs(
    title = "Adopción de herramientas de IA por profesión",
    subtitle = "Porcentaje de profesionistas que usan IA en su trabajo diario, 2025",
    x = NULL,
    y = NULL,
    caption = "Datos ficticios | Barras de error: IC al 95%"
  ) +
  theme_minimal(base_family = "Ubuntu") +
  theme(
    plot.title = element_text(face = "bold", size = 16, color = "gray10"),
    plot.subtitle = element_text(size = 11, color = "gray40", margin = margin(b = 15)),
    plot.caption = element_text(size = 8, color = "gray50", margin = margin(t = 15)),
    axis.text.y = element_text(size = 11, color = "gray20"),
    axis.text.x = element_blank(),
    panel.grid.major.y = element_blank(),
    panel.grid.major.x = element_line(color = "gray90", linewidth = 0.3),
    panel.grid.minor = element_blank(),
    plot.margin = margin(20, 20, 20, 20)
  )
p7
```

Adopción de herramientas de IA por profesión

Porcentaje de profesionistas que usan IA en su trabajo diario, 2025



Datos ficticios | Barras de error: IC al 95%

Partes del tema de una gráfica ggplot

ggplot2 theme elements reference

Set `minimal` as the baseline theme:

```
theme_minimal() +  
theme(theme.element = element_type())
```

Use `element_blank()` to remove an element

Axis titles, text, ticks, and lines can be specified per axis using theme inheritance by putting `.x/.y` at the end of the theme element.

```
axis.line.y = element_line()
```

```
axis.title.y = element_text()
```

```
panel.grid.major = element_line()
```

```
panel.grid.minor = element_line()
```

```
axis.text.y
```

```
axis.text = element_text()
```

axis.text.x

```
plot.title.position = "plot"
plot.caption.position = "plot"
plot.title = element_text()
plot.subtitle = element_text()
```

- "plot" means that they will be aligned to the entire plot (instead of the panel)

```
plot.margin = margin(25, 25, 25, 25)
```

```
legend.title  
= element_text()
```

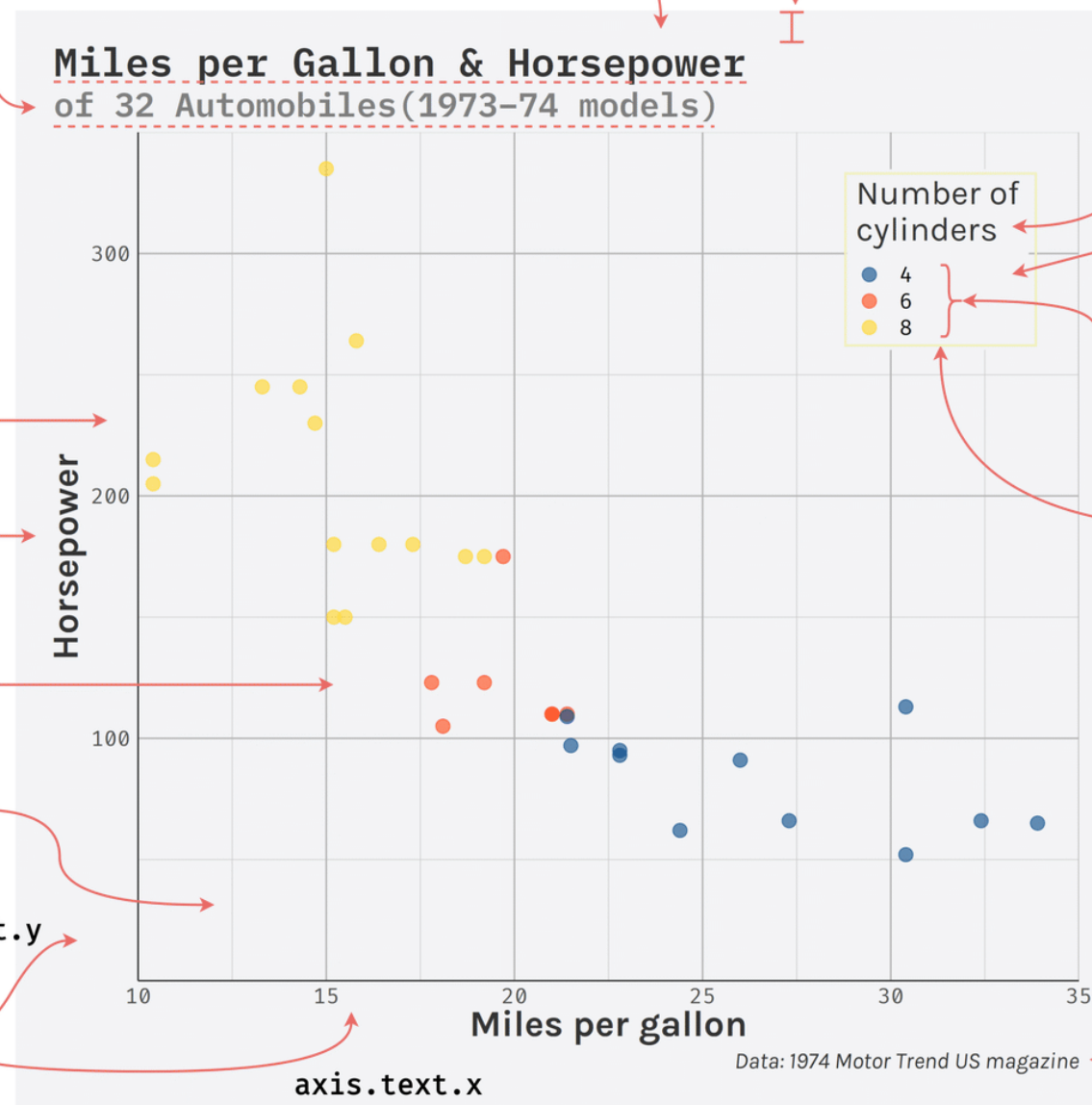
```
legend.background  
= element_rect()
```

```
legend.text  
= element_text()
```

```
legend.position
= c(.85,.85) / "none" /
"left" / "right" /
"bottom" / "top"
```

```
plot.background  
= element_rect()
```

```
plot.caption  
= element_text()
```



```
text = element_text() ← modifications will be applied to all text elements
```

Full list of elements at ggplot2.tidyverse.org/reference/theme

isabella-b

Partes del tema de una gráfica ggplot

ggplot2 Theme Elements

`theme(element_name = element_function())`

- `element_text()`
- `element_line()`
- `element_rect()`
- `element_blank()`

Axis elements:

`axis.ticks`
`element_line()`

`axis.title`
`element_text()`

`axis.text`
`element_text()`

`axis.line`
`element_line()`

Plot elements:

`plot.background`
`element_rect()`

`plot.title`
`element_text()`

`plot.margin`
`margin()`

Facetting elements:

`strip.background`
`element_rect()`

`panel.spacing`
`unit()`

`strip.text`
`element_text()`

Legend elements:

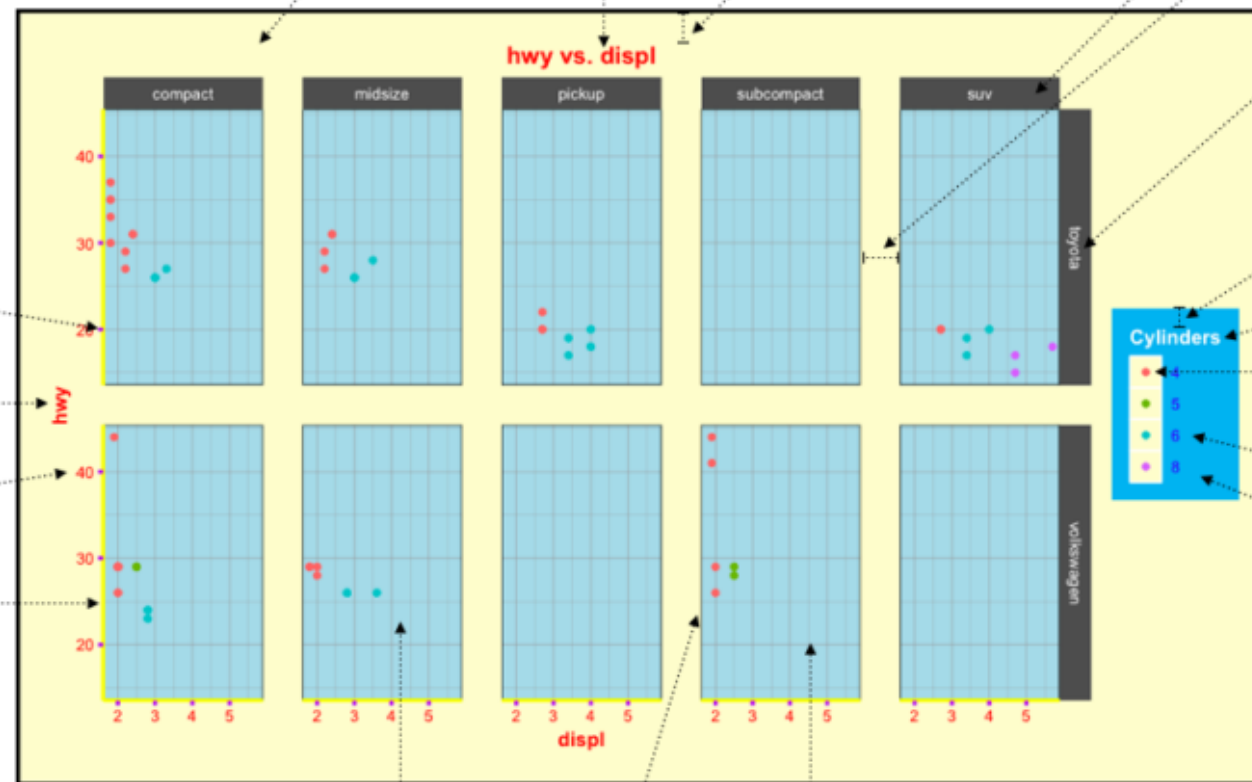
`legend.margin`
`margin()`

`legend.title`
`element_text()`

`legend.key`
`element_rect()`

`legend.text`
`element_text()`

`legend.background`
`element_rect()`



`panel.background`
`element_rect()`

`panel.grid`
`element_line()`

`panel.border`
`element_rect(fill = NA)`

Panel elements:

henrywang.nl

Derived from "ggplot2: Elegant Graphics for Data Analysis"

Ejercicio 02

Descargue el material de la sesión 04.

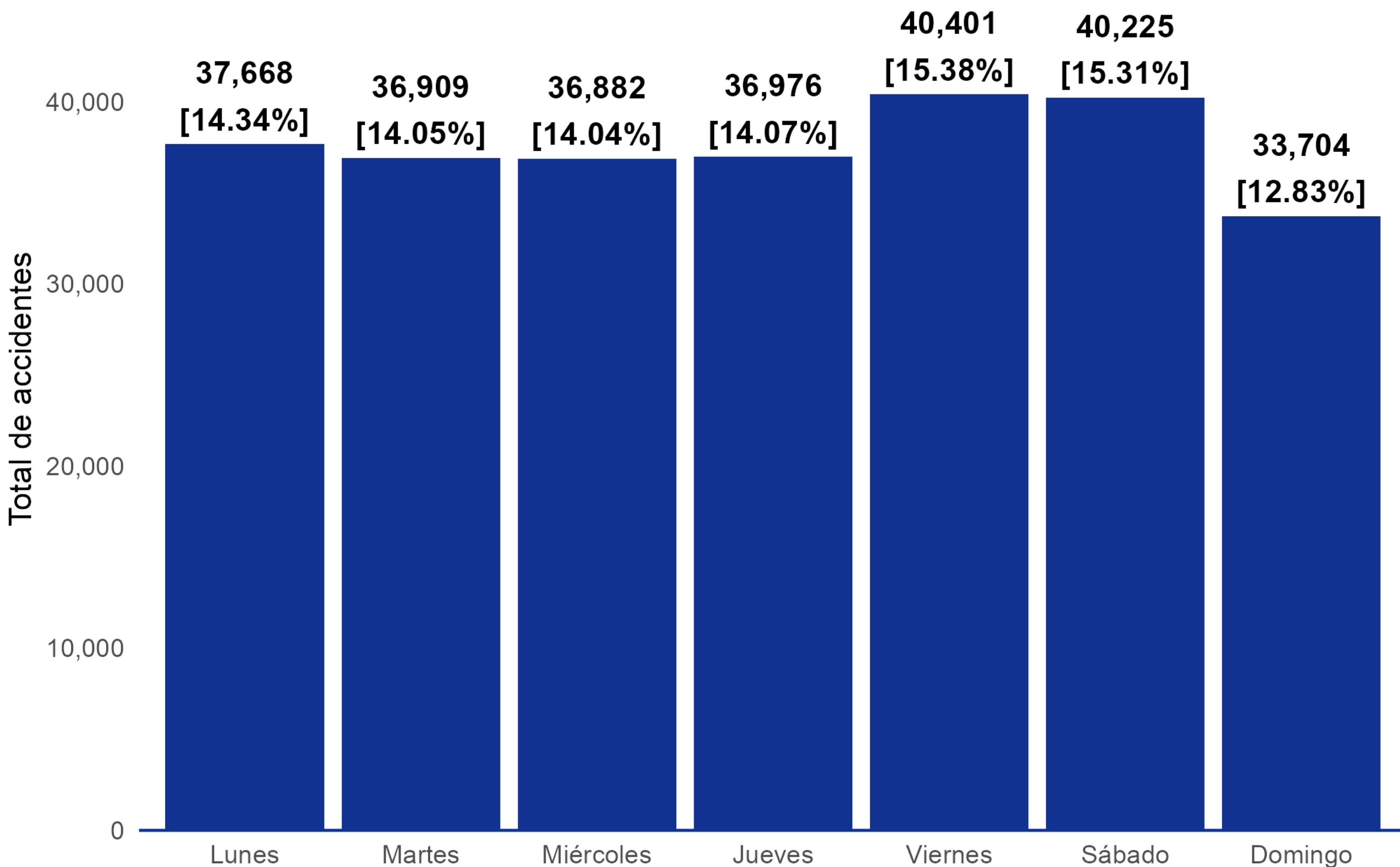
En la carpeta de “Ejercicio 02” viene la base de datos del ATUS (Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas). La unidad de observación de esta base de datos es el accidente. Si tiene dudas de la base, pregunte a su profesor.

Con esos datos, replique las gráficas que se ven a continuación en las diapositivas siguientes.

PLANTILLA

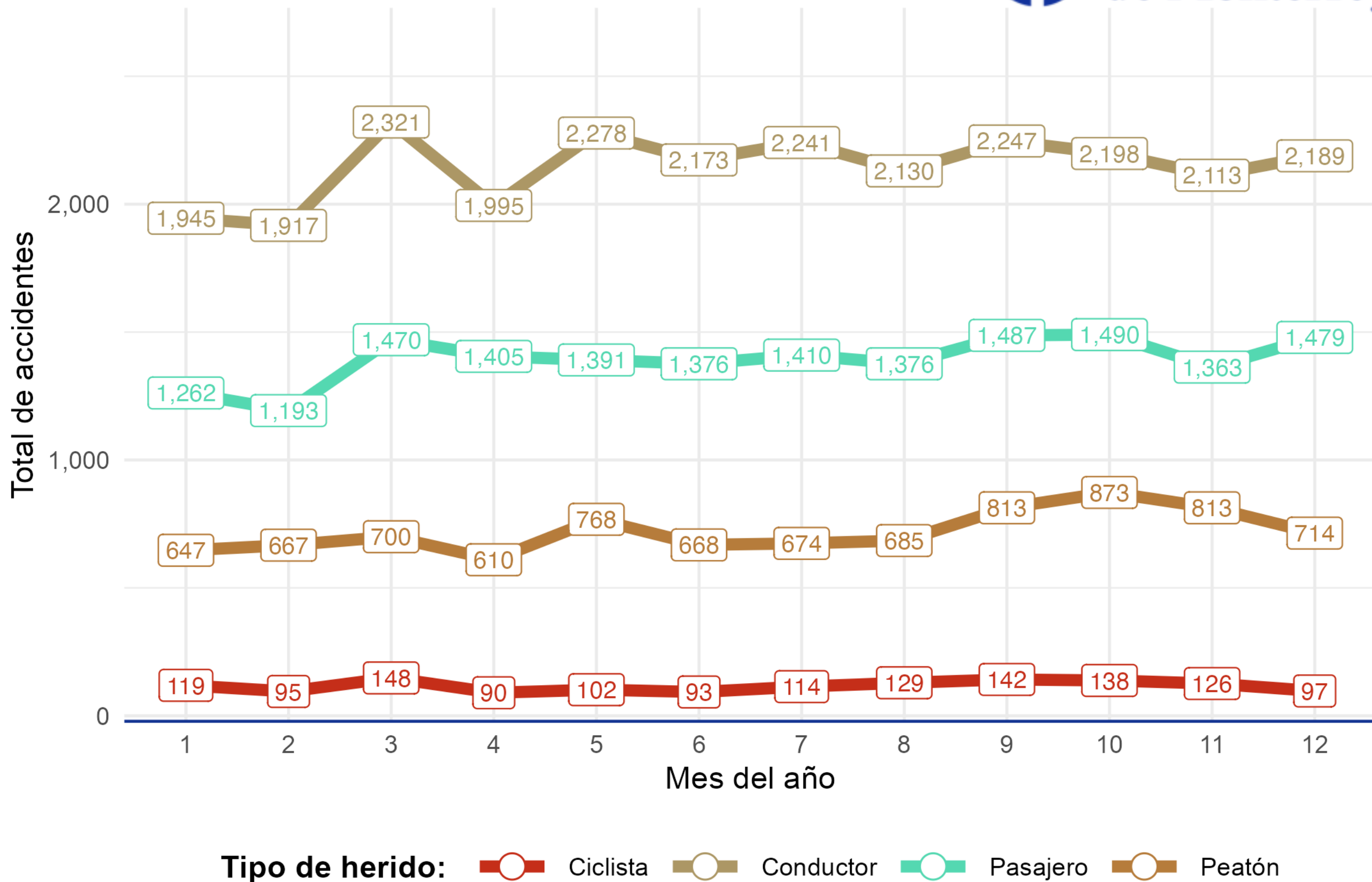
Total de accidentes por día de la semana

Accidentes correspondientes al año 2023



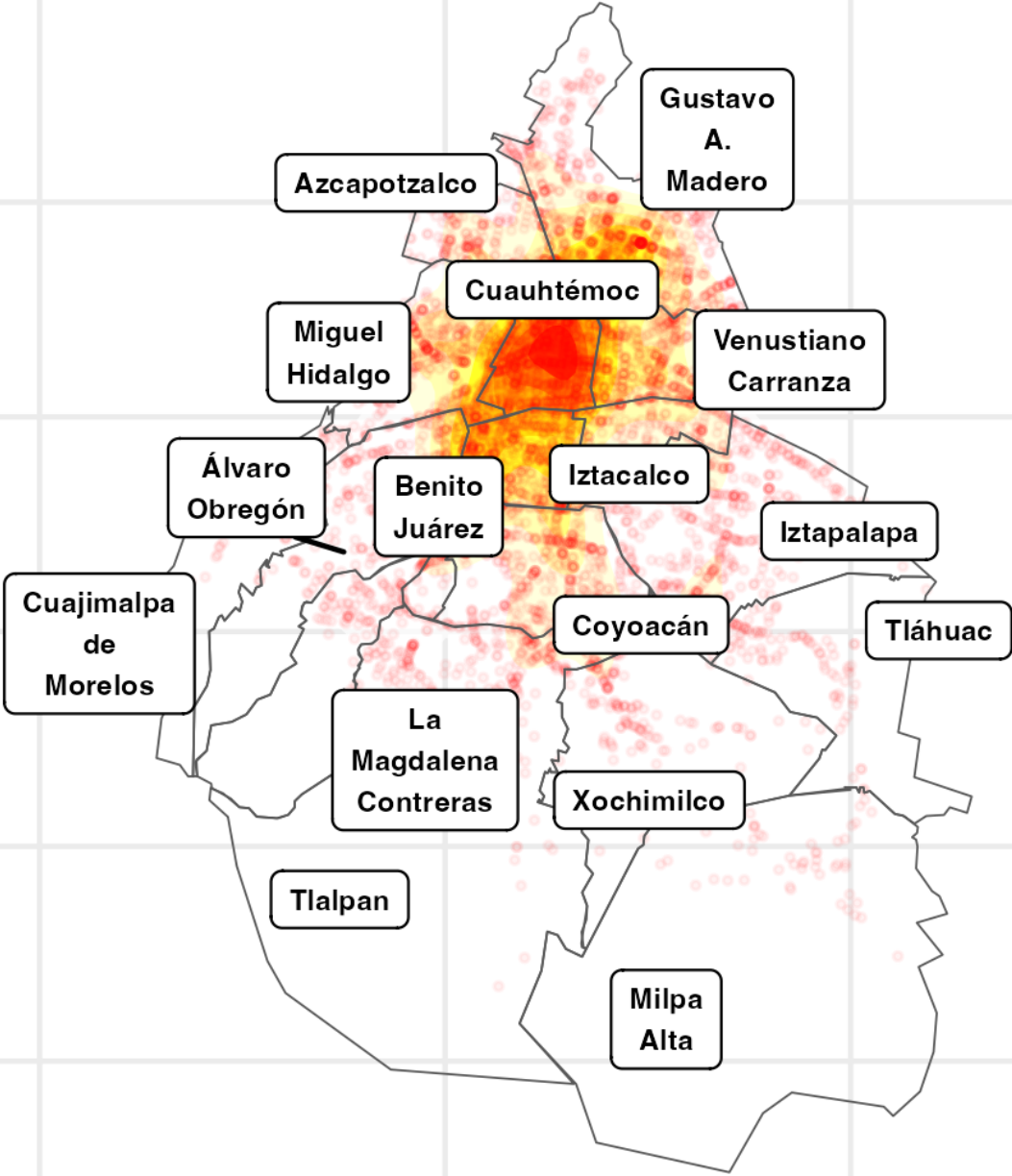
Total de heridos, por tipo y mes del año

Accidentes correspondientes al año 2023

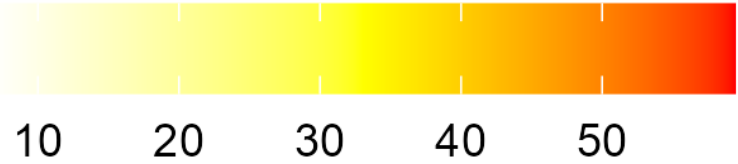


Ubicación de accidentes registrados en el ATUS en las alcaldías de la Ciudad de México

Accidentes correspondientes al año 2023

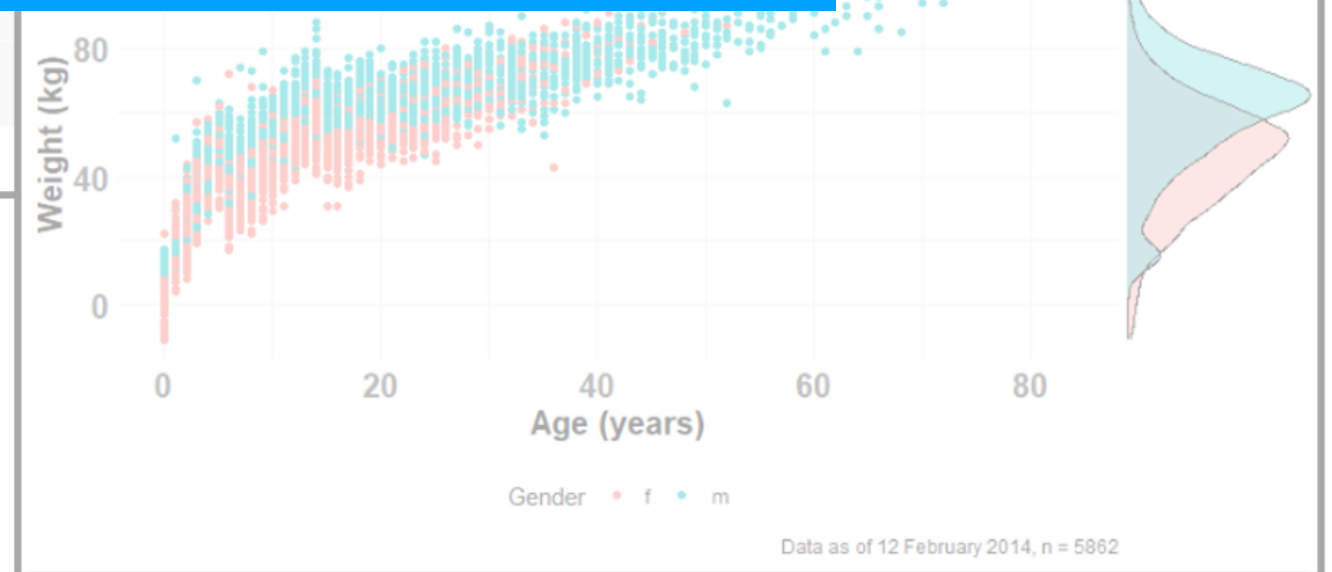


Densidad de accidentes





Diapositivas de apoyo



geom_col()

Crea gráficas de barras donde la altura de cada barra representa un valor ya calculado en los datos.

A diferencia de geom_bar(), NO cuenta frecuencias.

Sintaxis:

```
ggplot(datos, aes(x = categoria, y = valor)) +  
  geom_col()
```

Ejemplo:

```
library(tidyverse)  
  
# Datos de ventas por mes  
ventas <- tibble(  
  mes = c("Ene", "Feb", "Mar", "Abr"),  
  total = c(15000, 22000, 18000, 25000)  
)  
  
ggplot(ventas, aes(x = mes, y = total)) +  
  geom_col(fill = "#1A6BD4") +  
  labs(title = "Ventas mensuales")
```

Parámetros comunes

fill	Color de relleno de las barras
color	Color del borde de las barras
width	Ancho de las barras (0 a 1)
alpha	Transparencia (0 = invisible, 1 = sólido)
position	"dodge" para barras lado a lado, "stack" para apiladas

geom_col() vs geom_bar()

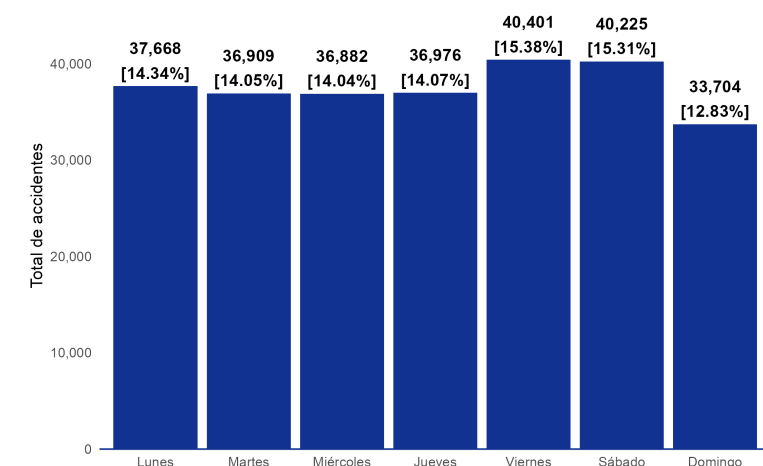
geom_col():

Usa los valores tal como están en los datos.
Requiere aes(x, y).

geom_bar():

Cuenta frecuencias automáticamente.
Solo necesita aes(x).

Total de accidentes por día de la semana
Accidentes correspondientes al año 2023



geom_point()

Crea gráficas de dispersión (*scatterplots*). Ideal para explorar relaciones entre dos variables numéricas.

Sintaxis:

```
ggplot(datos, aes(x = var_x, y = var_y)) +  
  geom_point()
```

Ejemplo:

```
library(tidyverse)  
  
# Relación entre cilindrada y rendimiento  
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point(  
    color = "#1B8A4A",  
    size = 3,  
    alpha = 0.7  
  ) +  
  labs(  
    title = "Cilindrada vs Rendimiento",  
    x = "Cilindrada (litros)",  
    y = "Millas por galón (autopista)"  
  )
```

Parámetros comunes

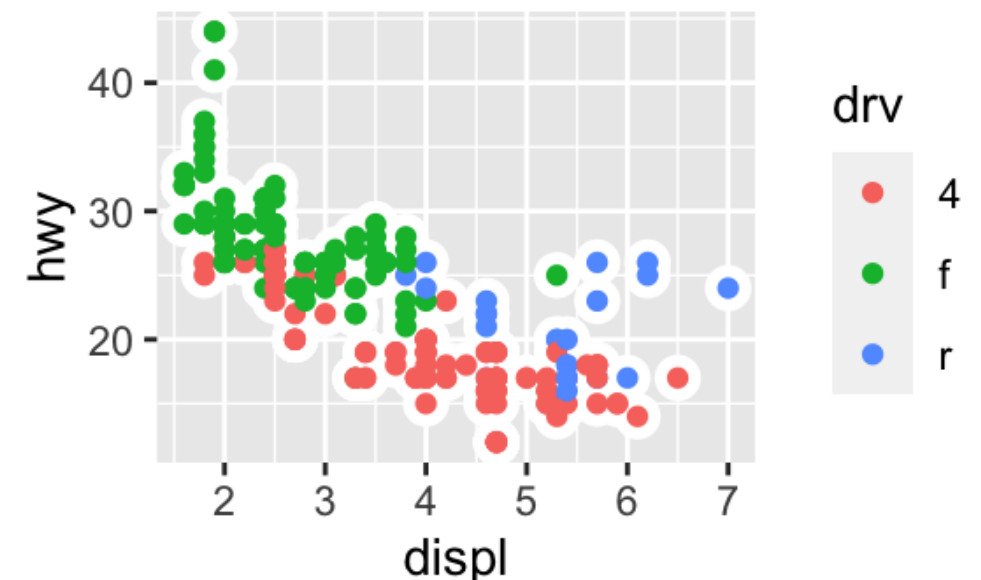
color	Color de los puntos
size	Tamaño de los puntos (default ~1.5)
shape	Forma: 16=círculo, 17=triángulo, 15=cuadrado...
alpha	Transparencia (útil si hay superposición)
stroke	Grosor del borde (para shapes con borde)

Mapeos estéticos dentro de aes()

```
# Color por categoría  
aes(color = clase)
```

```
# Tamaño por variable numérica  
aes(size = poblacion)
```

```
# Forma por grupo  
aes(shape = tipo)
```



geom_line()

Conecta observaciones con líneas, ordenadas por el eje X. Ideal para series de tiempo y datos con progresión.

Sintaxis:

```
ggplot(datos, aes(x = tiempo, y = valor)) +  
  geom_line()
```

Ejemplo:

```
library(tidyverse)  
  
# Serie de tiempo de temperatura  
temps <- tibble(  
  fecha = seq(as.Date("2025-01-01"),  
             as.Date("2025-12-01"), by = "month"),  
  temp = c(12,14,18,22,28,32,34,33,29,23,17,13)  
)  
  
ggplot(temps, aes(x = fecha, y = temp)) +  
  geom_line(color = "#C0397A", linewidth = 1.2) +  
  geom_point(color = "#C0397A", size = 3) +  
  labs(title = "Temperatura mensual 2025")
```

Parámetros comunes

color	Color de la línea
linewidth	Grosor de la línea (antes era 'size')
linetype	"solid", "dashed", "dotted", "dotdash"...
alpha	Transparencia de la línea
group	Agrupar líneas (dentro de aes())

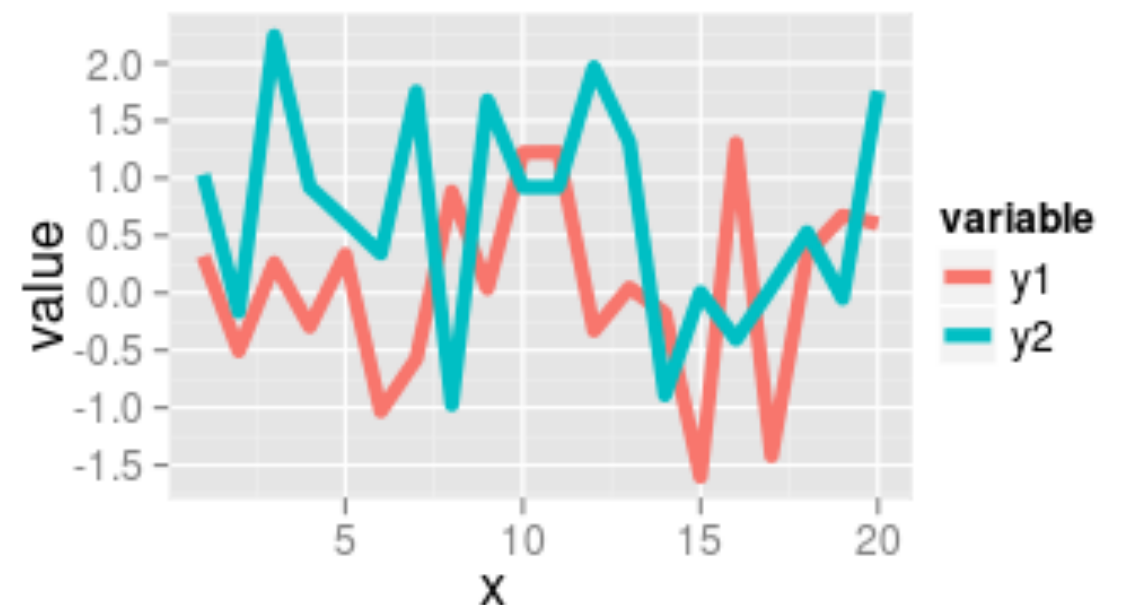
Tip: Combina geom_line + geom_point

Agregar puntos sobre la línea ayuda a identificar las observaciones exactas:

geom_line() + geom_point()

Para múltiples líneas, usa:

aes(color = grupo)



geom_sf()

Dibuja geometrías espaciales (mapas) a partir de objetos sf (Simple Features). Polígonos, puntos, líneas.

Sintaxis:

```
ggplot(datos_sf) +  
  geom_sf()
```

Ejemplo:

```
library(tidyverse)  
library(sf)  
  
# Leer un shapefile de estados de México  
mapa <- st_read("estados.shp")  
  
# Mapa coloreado por población  
ggplot(mapa) +  
  geom_sf(aes(fill = poblacion),  
          color = "white", linewidth = 0.2) +  
  scale_fill_viridis_c() +  
  theme_void() +  
  labs(title = "Población por estado")
```

Notas clave

Requiere el paquete sf instalado

Los datos deben ser un objeto sf con columna geometry

ggplot detecta automáticamente las coordenadas

fill colorea el interior de los polígonos

color controla el borde de los polígonos

Usa theme_void() para un mapa más limpio

scale_fill_viridis_c() es ideal para mapas

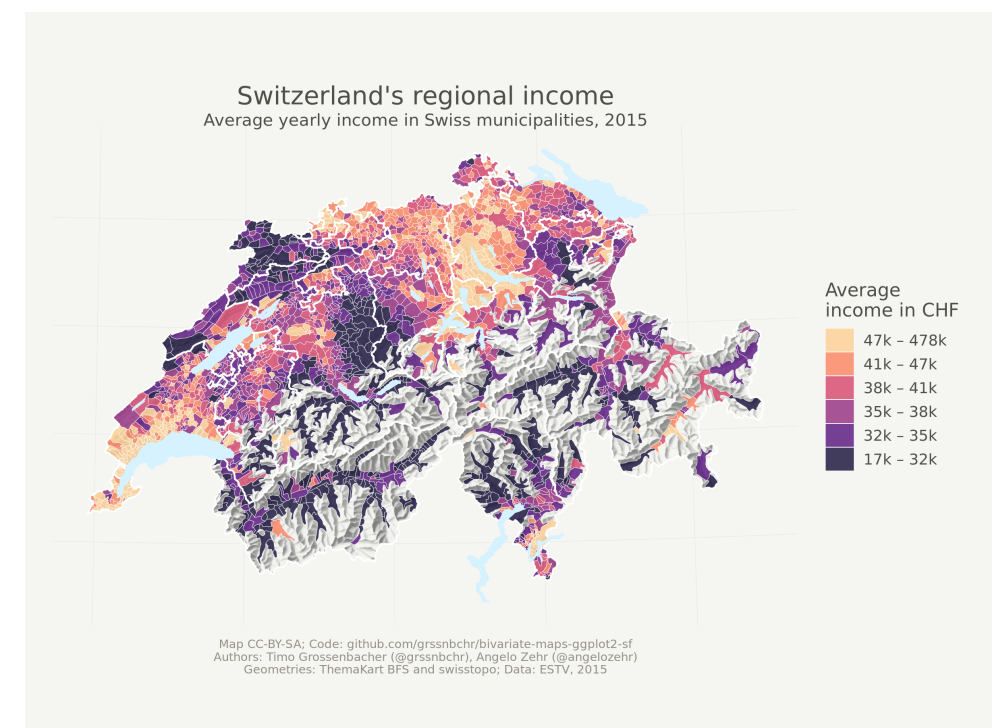
Fuentes de datos geográficos

.shp (Shapefile) — El más común

.geojson — Formato web, texto plano

.gpkg (GeoPackage) — Moderno, un solo archivo

Todos se leen con st_read() del paquete sf



mutate()

Crea nuevas columnas o modifica columnas existentes en un dataframe. Es uno de los verbos más usados del tidyverse. Las columnas nuevas se agregan al final del dataframe.

Sintaxis:

```
datos %>%
  mutate(nueva_col = expresion)
```

Ejemplo:

```
library(tidyverse)

# Crear nuevas columnas
mtcars %>%
  mutate(
    kpl = mpg * 0.425,      # millas a km/litro
    peso_ton = wt / 2.205,  # libras a toneladas
    potencia_cat = ifelse(
      hp > 150, "Alta", "Baja"
    )
  )
```

Notas clave

Siempre devuelve un dataframe del mismo número de filas

Puedes crear múltiples columnas en un solo mutate()

Las columnas nuevas pueden usar columnas recién creadas

Si el nombre ya existe, sobrescribe la columna

Dentro de mutate puedes usar: ifelse(), case_when(), str_c(), round(), y muchas más funciones

Para modificar solo un subconjunto, mira mutate() + if_else()

Tip

Usa .before o .after para controlar dónde se inserta la nueva columna:
mutate(nueva = x + y, .after = columna_ref)



case_when() e ifelse()

Funciones para crear valores condicionales dentro de mutate().

ifelse() — Dos opciones

```
# ifelse: condicion -> valor_si_TRUE, valor_si_FALSE
datos %>%
  mutate(
    categoria = ifelse(
      precio > 100,
      "Caro",
      "Barato"
    )
  )
```

case_when() — Múltiples casos

```
# case_when: multiples condiciones con ~
datos %>%
  mutate(
    nivel = case_when(
      calificacion >= 90 ~ "Excelente",
      calificacion >= 80 ~ "Bueno",
      calificacion >= 70 ~ "Regular",
      TRUE ~ "Insuficiente"
    )
  )
# TRUE es el "else" (todo lo demás)
```

Comparación

ifelse()

- + Simple: solo TRUE / FALSE
- + Rápido para decisiones binarias
- Se vuelve feo si anidas muchos

case_when()

- + Múltiples condiciones, limpio
- + Más legible que ifelse anidados
- + TRUE ~ captura "todo lo demás"
- Sintaxis con ~ puede confundir al inicio

¿Cuándo usar cuál?

2 opciones --> ifelse()

3+ opciones --> case_when()

Tip: También existe if_else() de dplyr, que es más estricto con los tipos de datos (recomendado).

dplyr::case_when() IF ELSE... (but you love it?)

df %>% ^{ADD column 'danger'} mutate(danger = case_when(^{IF type is kraken THEN} type == "kraken" ~ "extreme!", ^{T ~ "high"} TRUE ~ "high")) ^{OTHERWISE, danger is high.}

^{danger is extreme!}

type	age	danger
kraken	baby	extreme!
dragon	adult	high
cyclops	teen	high
kraken	adult	extreme!
dragon	teen	high

@alison_hart

clean_names()

Limpia los nombres de las columnas de un dataframe: los convierte a snake_case, elimina caracteres especiales, acentos, espacios y los hace consistentes.

Sintaxis:

```
library(janitor)

datos %>%
  clean_names()
```

Ejemplo:

```
library(tidyverse)
library(janitor)

# Datos con nombres feos
datos <- tibble(
  `Nombre Completo` = c("Ana", "Luis"),
  `Año de Nacimiento` = c(1995, 1998),
  `% Asistencia` = c(95.5, 88.2)
)

datos %>% clean_names()
# nombre_completo, ano_de_nacimiento,
# percent_asistencia
```

Notas clave

Convierte TODO a snake_case (minúsculas con _)

Elimina acentos: Año -> ano

Reemplaza espacios por guiones bajos

Convierte % a 'percent', # a 'number'

Elimina caracteres especiales

Si hay nombres duplicados, agrega sufijo numérico

Funciona con dataframes, tibbles y más

Úsalo siempre al importar datos externos

Tip

Ponlo justo después de leer tus datos:
read_csv('archivo.csv') %>% clean_names()



str_c()

Concatena (pega) cadenas de texto. Es la versión tidyverse de paste0(). Más predecible con valores NA que paste().

Sintaxis:

```
str_c("texto1", "texto2", sep = " ")
```

Ejemplo:

```
library(tidyverse)

# Concatenar columnas
datos <- tibble(
  nombre = c("Ana", "Luis"),
  apellido = c("García", "López")
)

datos %>%
  mutate(
    nombre_completo = str_c(nombre, apellido,
                           sep = " "),
    saludo = str_c("Hola, ", nombre, "!")
  )
# "Ana García", "Hola, Ana!"
```

Notas clave

sep = define el separador entre elementos

collapse = une un vector en un solo string

Si algún valor es NA, el resultado es NA (a diferencia de paste que pone "NA")

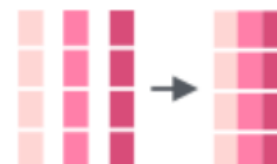
str_c('a', 'b') -> 'ab' (sin separador)

str_c('a', 'b', sep='-') -> 'a-b'

Equivalente a paste0() pero más consistente

Tip

Usa collapse para colapsar un vector:
str_c(c("a", "b", "c"), collapse=", ") -> "a, b, c"



str_c(..., sep = "", collapse = NULL) Join multiple strings into a single string.
str_c(letters, LETTERS)

round()

Redondea números al número de decimales especificado. Función base de R, no requiere paquetes adicionales.

Sintaxis:

```
round(x, digits = 0)
```

Ejemplo:

```
# Redondeo básico
round(3.14159, 2)  # 3.14
round(3.14159, 0)  # 3
round(1234.5, -2)  # 1200

# Dentro de mutate
datos %>%
  mutate(promedio = round(mean(valor), 2),
         porcentaje = round(n / total * 100, 1))
```

Notas clave

digits = 0 -> sin decimales (entero)

digits = 2 -> dos decimales

digits negativo -> redondea a decenas, centenas, etc.

round(1250, -2) -> 1200

Funciones relacionadas:

ceiling() -> redondea hacia arriba

floor() -> redondea hacia abajo

trunc() -> elimina decimales

R usa 'banker's rounding': round(2.5) = 2
(redondea al par más cercano en empates)

ungroup()

Elimina la agrupación de un dataframe creada con `group_by()`. Es fundamental para evitar cálculos inesperados en operaciones posteriores.

Sintaxis:

```
datos %>%  
  group_by(variable) %>%  
  summarise(...) %>%  
  ungroup()
```

Ejemplo:

```
library(tidyverse)  
  
# Sin ungroup: el mutate opera por grupo  
mtcars %>%  
  group_by(cyl) %>%  
  summarise(promedio = mean(mpg)) %>%  
  ungroup() %>% # <- Importante!  
  mutate(porcentaje = promedio / sum(promedio) * 100)  
  
# Sin ungroup, sum(promedio) seria por grupo  
# Con ungroup, sum(promedio) es el total global
```

Notas clave

`group_by()` se 'hereda' en todas las operaciones posteriores hasta que hagas `ungroup()`

`summarise()` quita UN nivel de agrupación, pero si tenías `group_by(a, b)`, queda agrupado por 'a'

Buena práctica: siempre usar `ungroup()` después de terminar operaciones agrupadas

Si no desagrupas, `mutate()`, `filter()` y otras funciones operarán dentro de cada grupo

Puedes verificar con: `groups(datos)`

Tip

Regla de oro: si usas `group_by()`, siempre termina con `ungroup()` cuando ya no lo necesites.

aes() — Dentro vs Fuera del geom

La ubicación de los parámetros estéticos cambia completamente el comportamiento de la gráfica.

Dentro de aes() — Mapeo a datos

```
# El COLOR depende de una variable  
ggplot(datos, aes(x = mes, y = ventas)) +  
  geom_col(aes(fill = region))
```

```
# Cada región tendrá un color diferente  
# ggplot asigna colores automáticamente
```

El valor cambia según los datos

Se genera una leyenda automáticamente

Usa nombres de columnas (sin comillas)

Ejemplo: `aes(fill = region)`

-> Norte=azul, Sur=rojo, Centro=verde...

Fuera de aes() — Valor fijo

```
# TODAS las barras del mismo color  
ggplot(datos, aes(x = mes, y = ventas)) +  
  geom_col(fill = "#1A6BD4")
```

```
# Un solo color fijo para TODO el geom  
# No se genera leyenda
```

Aplica el MISMO valor a todos los elementos

No genera leyenda

Usa valores literales (con comillas para colores)

Ejemplo: `fill = "#1A6BD4"`

-> Todas las barras: azul

base R

prettyNum()

Formatea números para presentación: agrega separadores de miles, controla decimales y devuelve texto legible. Ideal para etiquetas en gráficas y tablas.

Sintaxis:

```
prettyNum(x, big.mark = ",", scientific = FALSE)
```

Ejemplo:

```
# Formato con comas
prettyNum(1234567, big.mark = ",")
# "1,234,567"

# Dentro de mutate para etiquetas
datos %>%
  mutate(etiqueta = prettyNum(poblacion, big.mark = ",", scientific = FALSE))

# En gráficas
geom_text(aes(label = prettyNum(valor, big.mark = ",")))
```

Notas clave

`big.mark = ","` -> separador de miles con coma

`big.mark = "."` -> separador de miles con punto (como en España)

`scientific = FALSE` -> evita notación científica

Devuelve un CHARACTER, no un número

Muy útil para `geom_text()` y `geom_label()`

Para más control, revisa `scales::comma()` y `format()`

Tip

Para formato mexicano:
`prettyNum(1234567, big.mark = ",")` -> "1,234,567"



Medidor de color digital (Color Picker)

Herramienta para obtener el código hexadecimal exacto de cualquier color en pantalla.

¿Qué es un código hexadecimal?

Formato: #RRGGBB

Cada par de letras = intensidad de Rojo, Verde, Azul
Rango: 00 (nada) a FF (máximo)

	#FF0000	Rojo puro
	#00AA00	Verde
	#0000FF	Azul puro
	#003DA5	Azul Tec
	#C0397A	Rosa

¿Cómo obtener colores?

En macOS:

Utilidades -> Medidor de color digital
O buscar "Digital Color Meter" en Spotlight

En Windows:

PowerToys -> Color Picker (Win+Shift+C)

En la web:

Google: buscar "color picker"
colors.co — paletas de colores

Uso en ggplot2

```
# Usar el código hex directamente
geom_col(fill = "#003DA5")
geom_point(color = "#C0397A")

# Para múltiples colores manuales
scale_fill_manual(
  values = c("#003DA5", "#1B8A4A",
             "#C0397A", "#D06B21")
)
```

ggsave()

Guarda la última gráfica creada con ggplot2 en un archivo. Detecta el formato automáticamente por la extensión del archivo. Permite controlar tamaño, resolución y más.

Sintaxis:

```
ggsave("nombre.png", width = 10, height = 6, dpi = 300)
```

Ejemplo:

```
library(tidyverse)

# Crear gráfica
mi_grafica <- ggplot(mpg, aes(displ, hwy)) +
  geom_point()

# Guardar como PNG (alta resolución)
ggsave("grafica.png", plot = mi_grafica, width = 10, height = 6, dpi = 300)

# Guardar como SVG (vectorial)
ggsave("grafica.svg", width = 10, height = 6)
```

Notas clave

Si no especificas plot =, guarda la ÚLTIMA gráfica

width y height están en pulgadas por defecto

units = 'cm' o 'mm' para cambiar unidades

dpi = 300 es estándar para impresión

dpi = 72 es suficiente para pantalla/web

Formatos soportados: png, jpeg, svg, pdf, tiff, bmp

Para guardar en carpeta:

```
ggsave("carpeta/grafica.png")
```

Tip

Para presentaciones usa width=16, height=9 (16:9).

Para redes sociales: width=10, height=10 (cuadrado).



PNG vs JPEG vs SVG

Cada formato tiene ventajas y desventajas. Elegir el correcto depende del uso.

PNG

Portable Network Graphics



Tipo: Raster (píxeles)

Compresión: Sin pérdida

Transparencia: Sí (canal alpha)

Ventajas:

- Calidad perfecta, sin artefactos
- Soporta transparencia
- Ideal para gráficas y diagramas

Desventajas:

- Archivos más pesados que JPEG
- Pierde calidad al escalar

Usar para:

Gráficas, capturas, logos, web

JPEG

Joint Photographic Experts Group



Tipo: Raster (píxeles)

Compresión: Con pérdida

Transparencia: No soporta

Ventajas:

- Archivos muy ligeros
- Excelente para fotografías
- Compatible con todo

Desventajas:

- Pierde calidad (artefactos)
- Sin transparencia

Usar para:

Fotos, imágenes con muchos colores, web si el peso importa

SVG

Scalable Vector Graphics



Tipo: Vectorial (matemáticas)

Compresión: No aplica

Transparencia: Sí

Ventajas:

- Escala infinitamente sin perder calidad
- Archivos pequeños para gráficas
- Editable (es código XML)

Desventajas:

- No ideal para fotos
- Puede ser pesado si es complejo

Usar para:

Gráficas para publicación, logos, íconos, presentaciones

Recomendación para gráficas en R: PNG (uso general, dpi=300) | SVG (publicaciones, máxima calidad) | JPEG (solo si necesitas archivos muy ligeros)

Named Vectors (Vectores con nombre)

Un *named vector* es un vector donde cada elemento tiene un nombre asociado. Esto permite acceder a valores por nombre en lugar de por posición, y es clave para mapear valores en ggplot2.

Sintaxis:

```
mi_vector <- c("nombre_1" = valor1,
               "nombre_2" = valor2,
               "nombre_3" = valor3)
```

Ejemplos:

```
# Definir colores para categorías
colores <- c("Norte" = "#003DA5",
             "Centro" = "#1B8A4A",
             "Sur" = "#C0397A")

# Usar en ggplot con scale_*_manual()
ggplot(datos, aes(x = region, y = valor,
                  fill = region)) +
  geom_col() +
  scale_fill_manual(values = colores)

# Acceder por nombre
colores["Norte"] # "#003DA5"
```

¿Para qué sirven?

Asignar colores fijos a categorías en `scale_fill_manual()` / `scale_color_manual()`

Renombrar o recodificar valores con `recode()` o `deframe()`

Acceder a valores por nombre en vez de por posición numérica

Crear etiquetas personalizadas para facetas o ejes en gráficas

Definir parámetros de configuración reutilizables en tu script

Uso típico en ggplot2

El nombre del vector debe coincidir exactamente con los valores de la variable en los datos:

```
# Si la columna 'region' tiene: Norte, Centro, Sur
# Los nombres del vector deben ser iguales:
c("Norte" = "#003DA5", "Centro" = "#1B8A4A",
  "Sur" = "#C0397A")
```